Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka Techniczna

Badanie efektywności wybranych algorytmów sortowania ze względu na złożoność obliczeniową

Imię i nazwisko: Wiktor Zięba
Numer albumu: 281056

Wrocław, kwiecień 2025

Spis treści

1 Plan eksperymentu **3**

2 Sortowanie przez wstawianie **4**
2.1 Wprowadzenie . . . 4
2.2 Wykresy przedstawiające uzyskane wyniki . . . 5
2.2.1 Insertion sort dla tablicy losowej . . . 5
2.2.2 Insertion sort dla tablicy posortowanej rosnąco . . . 5
2.2.3 Insertion sort dla tablicy posortowanej malejąco . . . 6
2.2.4 Insertion sort dla tablicy posortowanej w 66% . . . 6
2.2.5 Insertion sort dla tablicy posortowanej w 33% . . . 7
2.2.6 Porównanie algorytmu insertion sort w zależności od tablicy początkowej . . . 7
2.3 Wnioski . . . 8

3 Sortowanie Shell'a **9**
3.1 Wprowadzenie . . . 9
3.2 Wykresy przedstawiające uzyskane wyniki . . . 10
3.2.1 Shellsort - tablica losowa . . . 10
3.2.2 Shellsort - tablica posortowana rosnąco . . . 10
3.2.3 Shellsort - tablica posortowana malejąco . . . 11
3.2.4 Shellsort - tablica posortowana w 66% . . . 11
3.2.5 Shellsort - tablica posortowana w 33% . . . 12
3.2.6 Shellsort(knuth) - tablica losowa . . . 12
3.2.7 Shellsort(knuth) - tablica posortowana rosnąco . . . 13
3.2.8 Shellsort(knuth) - tablica posortowana malejąco . . . 13
3.2.9 Shellsort(knuth) - tablica posortowana w 66% . . . 14
3.2.10 Shellsort(knuth) - tablica posortowana w 33% . . . 14
3.2.11 Porównanie dla tablicy losowej . . . 15
3.2.12 Porównanie dla tablicy posortowanej rosnąco . . . 15
3.2.13 Porównanie dla tablicy posortowanej malejąco . . . 16
3.2.14 Porównanie dla tablicy posortowanej w 66% . . . 16
3.2.15 Porównanie dla tablicy posortowanej w 33% . . . 17
3.3 Wnioski . . . 17

4 Sortowanie szybkie **18**
4.1 Wprowadzenie . . . 18
4.2 Wykresy przedstawiające uzyskane wyniki . . . 19
4.2.1 Quicksort dla tablicy wypełnionej losowymi liczbami . . . 19
4.2.2 Quicksort dla tablicy posortowanej rosnąco . . . 19
4.2.3 Quicksort dla tablicy posortowanej malejąco . . . 20
4.2.4 Quicksort dla tablicy posortowanej w 66% . . . 20
4.2.5 Quicksort dla tablicy posortowanej w 33% . . . 21
4.3 Porównanie pivotów w algorytmie quicksort oraz wykresy . . . 21
4.3.1 Dla tablicy losowej . . . 21
4.3.2 Dla tablicy posortowanej rosnąco . . . 22
4.3.3 Dla tablicy posortowanej malejąco . . . 22
4.3.4 Dla tablicy posortowanej w 66% . . . 23
4.3.5 Dla tablicy posortowanej w 33% . . . 23
4.4 Wnioski . . . 23

5 Sortowanie przez kopcowanie **25**
5.1 Wprowadzenie . . . 25
5.2 Wykresy przedstawiające uzyskane wyniki . . . 26
5.2.1 Heapsort dla tablicy wypełnionej losowymi liczbami . . . 26
5.2.2 Heapsort dla tablicy posortowanej rosnąco . . . 26
5.2.3 Heapsort dla tablicy posortowanej malejąco . . . 27
5.2.4 Heapsort dla tablicy posortowanej w 66% . . . 27
5.2.5 Heapsort dla tablicy posortowanej w 33% . . . 28
5.2.6 Heapsort - porównanie szybkości algorytmu na różnych typach tablic . . . 28
5.3 Wnioski . . . 29

6 Porównanie wpływu typu danych na szybkość sortowania na podstawie sortowania przez kopcowanie **30**
6.1 Wprowadzenie . . . 30
6.2 Wykresy przedstawiające uzyskane wyniki . . . 30
6.2.1 Dla tablicy losowej . . . 30
6.2.2 Dla tablicy posortowanej rosnąco . . . 31
6.2.3 Dla tablicy posortowanej malejąco . . . 31
6.2.4 Dla tablicy posortowanej w 66% . . . 32
6.2.5 Dla tablicy posortowanej w 33% . . . 32
6.3 Wnioski . . . 32

7 Porównanie ogólne wszystkich algorytmów **33**
7.1 Wprowadzenie . . . 33
7.2 Wykresy przedstawiające uzyskane wyniki . . . 33
7.2.1 Porównanie wszystkich przedstawionych w sprawozdaniu algorytmów . . . 33
7.2.2 Dla tablicy losowej . . . 34
7.2.3 Dla tablicy uporządkowanej rosnąco . . . 34
7.2.4 Dla tablicy uporządkowanej malejąco . . . 35
7.2.5 Dla tablicy uporządkowanej w 66% . . . 35
7.2.6 Dla tablicy uporządkowanej w 33% . . . 36
7.3 Wnioski . . . 36

8 Literatura **37**

1 Plan eksperymentu

Zadanie projektowe polegało na wielokrotnym przetestowaniu czterech różnych algorytmów sortowania oraz pomiaru czasu sortowania tablic w funkcji ich rozmiaru. Przedmiotem badań były następujące algorytmy:

- Insertion Sort
- Quick Sort
- Shell Sort
- Heap Sort

Dla każdego algorytmu dla zwiększenia wiarygodności pomiarów przeprowadzono 100 powtórzeń. Działanie każdego z algorytmów zostało przetestowane na 7 różnych rozmiarach tablic odpowiednio kolejno 40000, 60000, 80000, 100000, 120000, 140000 elementów. Dla każdego rozmiaru, tablice generowano w pięciu wariantach początkowego ułożenia danych:

- Tablicy losowej (całość elementów tablicy została wygenerowana losowo)
- Tablicy posortowanej rosnąco
- Tablicy posortowanej malejąco
- Tablicy posortowanej rosnąco w 33% (pierwsze 33% elementów tablicy wynosi 1)
- Tablicy posortowanej rosnąco w 66% (pierwsze 66% elementów tablicy wynosi 1)

Każdy algorytm dla zwiększenia wiarygodności pomiarów pracował na kopii tej samej tablicy, w każdej iteracji po przetestowaniu wszystkich algorytmów tablica generowała się na nowo. Oprócz klasycznych pomiarów, dodatkowo zbadano wpływ typu danych (int, float, double) na czas działania algorytmu. Sprawdzono także wpływ parametrów algorytmów, takich jak:

- sposób wyboru pivota w QuickSort (środkowy, losowy, skrajny lewy i prawy)
- sposób wyznaczania odstępów w ShellSort (formuła SHELL i KNUTH)

Wszelkie pomiary czasów sortowania poszczególnych algorytmów zostały zrealizowane przy użyciu biblioteki `std::chrono::high_resolution_clock` zapewniającej wysoką precyzję uzyskanych wyników. Pomiary obejmowały jedynie czasy wykonania algorytmów sortowania z pominięciem np. generowania danych lub sprawdzania poprawności. Wszystkie testy przeprowadzone zostały na tym samym komputerze w celu zapewnienia spójnych warunków pomiarowych.

2 Sortowanie przez wstawianie

2.1 Wprowadzenie

Sortowanie przez wstawianie (Insertion Sort) to jeden z najprostszych algorytmów sortujących. Polega na przeglądaniu elementów tablicy jeden po drugim i wstawianiu każdego z nich w odpowiednie miejsce względem już posortowanej części. Algorytm porównuje każdy nowy element z poprzednimi i przesuwa go w lewo tak długo, aż znajdzie odpowiednie miejsce. W każdej iteracji rośnie część tablicy, która jest posortowana, a maleje część nieuporządkowana. Algorytm ten jest stabilny – nie zmienia kolejności elementów o tej samej wartości – oraz działa w miejscu (in-place), tzn. nie wymaga dodatkowych struktur danych. Sortowanie odbywa się na tej samej tablicy, na której pracuje algorytm. Poniżej przedstawiono uproszczony pseudokod algorytmu:

```
i := 1
while i < length(A)
    j := i
    while j > 0 and A[j-1] > A[j]
        swap A[j] and A[j-1]
        j := j - 1
    end while
    i := i + 1
end while
```

Tekst 1: Insertion Sort pseudokod

Poniższa tabela przedstawia złożoność obliczeniową algorytmu Insertion Sort - czasową jak i pamięciową. W najlepszym przypadku złożoność czasowa wynosi O(n), dzieje się tak gdy wejściowe dane były już posortowane rosnąco. Najgorszą złożoność czasową O(n^2) algorytm osiąga dla wejściowych danych posortowanych odwrotnie, wtedy algorytm każdy element musi przesunąć na początek tablicy.

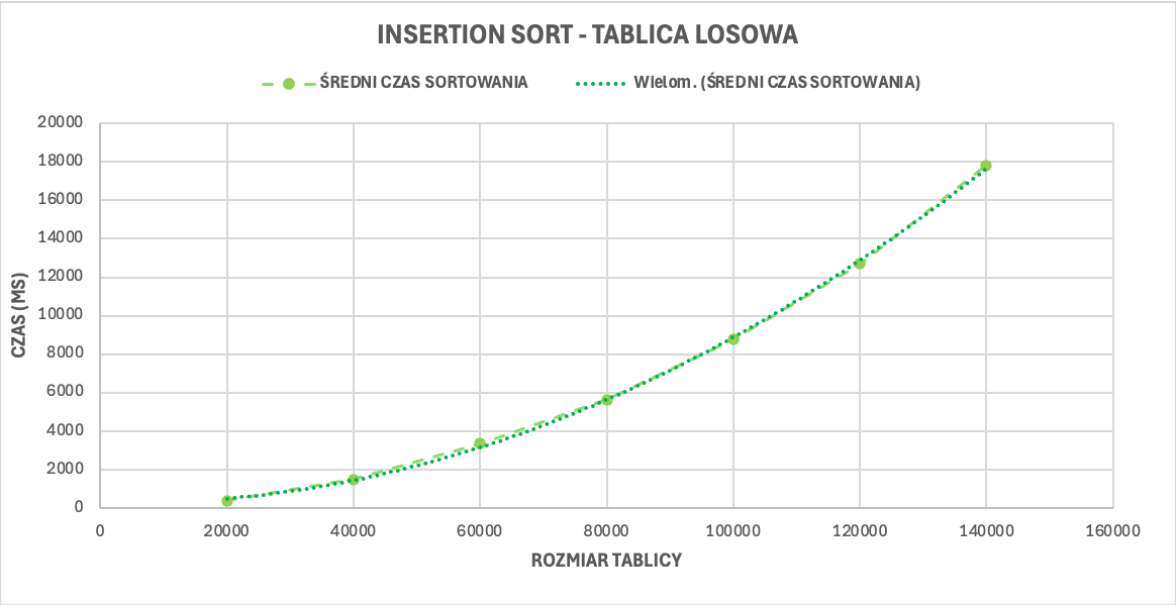
Tabela 1: Podsumowanie złożoności czasowej i pamięciowej algorytmu Insertion Sort [2]

Przypadek	**Złożoność**
Najlepszy przypadek (czasowa)	$O(n)$
Najgorszy przypadek (czasowa)	$O(n^2)$
Średni przypadek (czasowa)	$O(n^2)$
Złożoność pamięciowa	$O(1)$

Istnieje również wariant algorytmu z wyszukiwaniem binarnym w którym zamiast liniowego przeszukiwania miejsca wstawienia wykorzystywana jest wyszukiwanie binarne. Pozwala to zmniejszyć liczbę porównań do $O(n \log n)$, jednak nie wpływa na złożoność ogólną algorytmu.

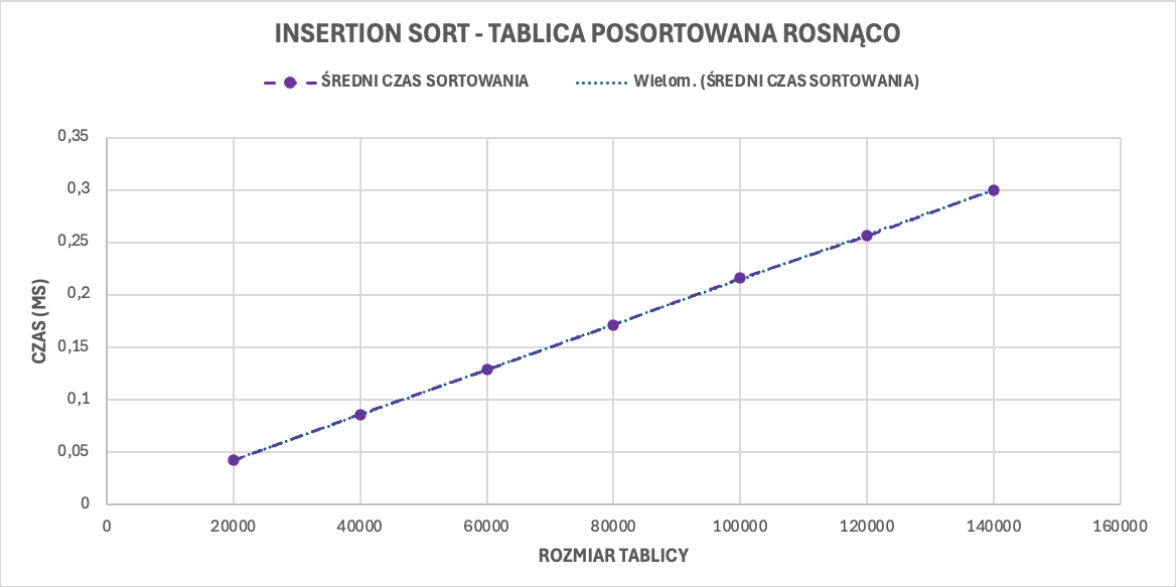
2.2 Wykresy przedstawiające uzyskane wyniki

2.2.1 Insertion sort dla tablicy losowej



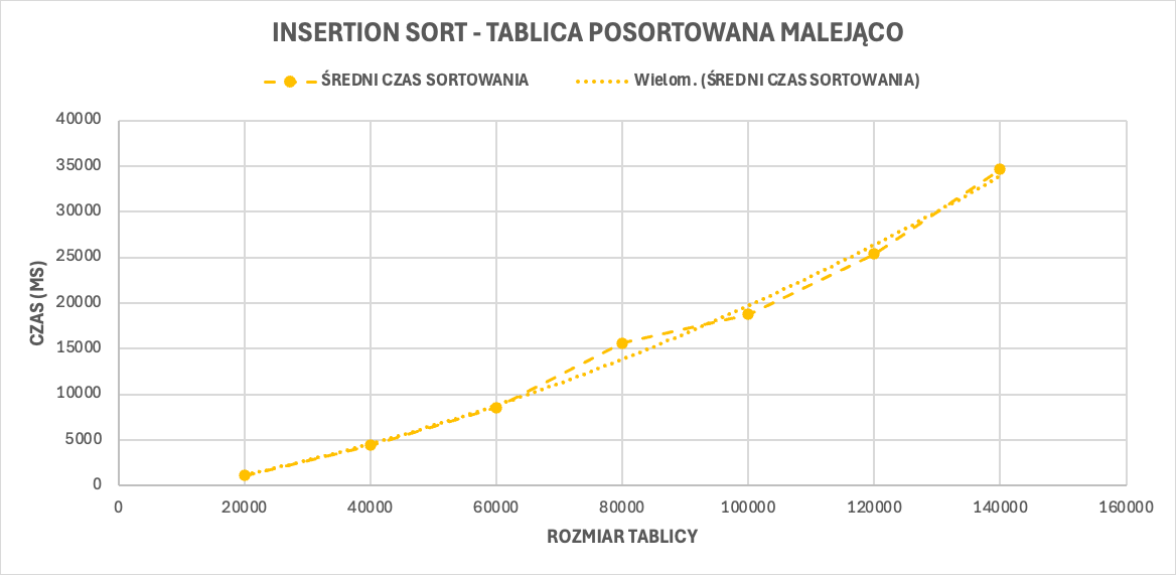
Wykres 1: Wykres czasu sortowania Insertion Sort w funkcji rozmiaru tablicy - tablica losowa

2.2.2 Insertion sort dla tablicy posortowanej rosnąco



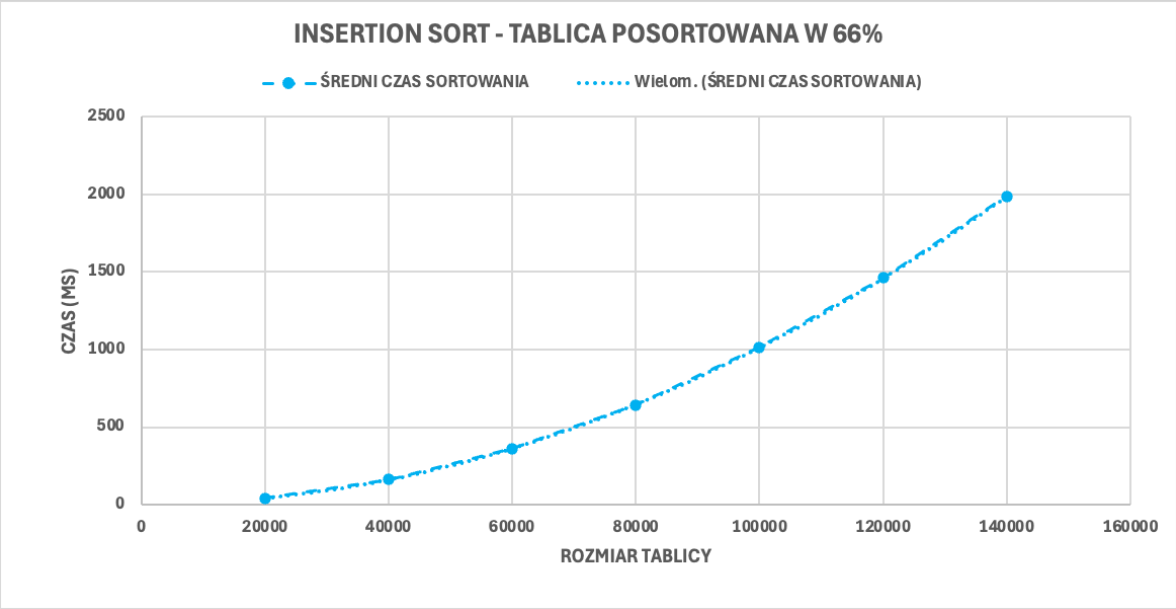
Wykres 2: Wykres czasu sortowania Insertion Sort w funkcji rozmiaru tablicy - tablica posortowana rosnąco

2.2.3 Insertion sort dla tablicy posortowanej malejąco



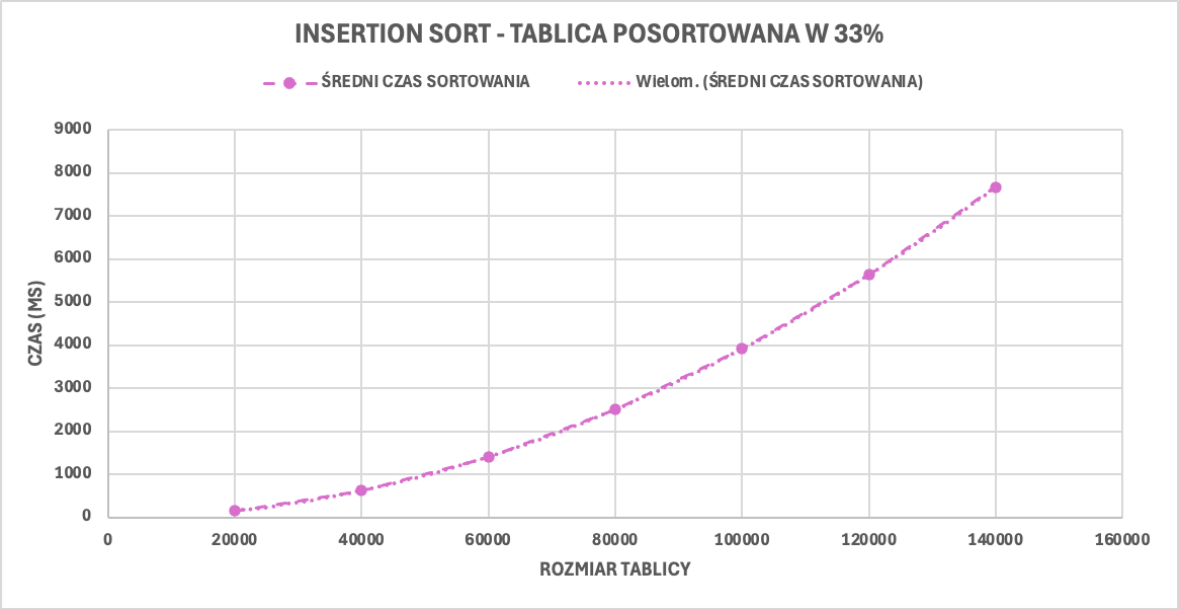
Wykres 3: Wykres czasu sortowania Insertion Sort w funkcji rozmiaru tablicy - tablica posortowana malejąco

2.2.4 Insertion sort dla tablicy posortowanej w 66%



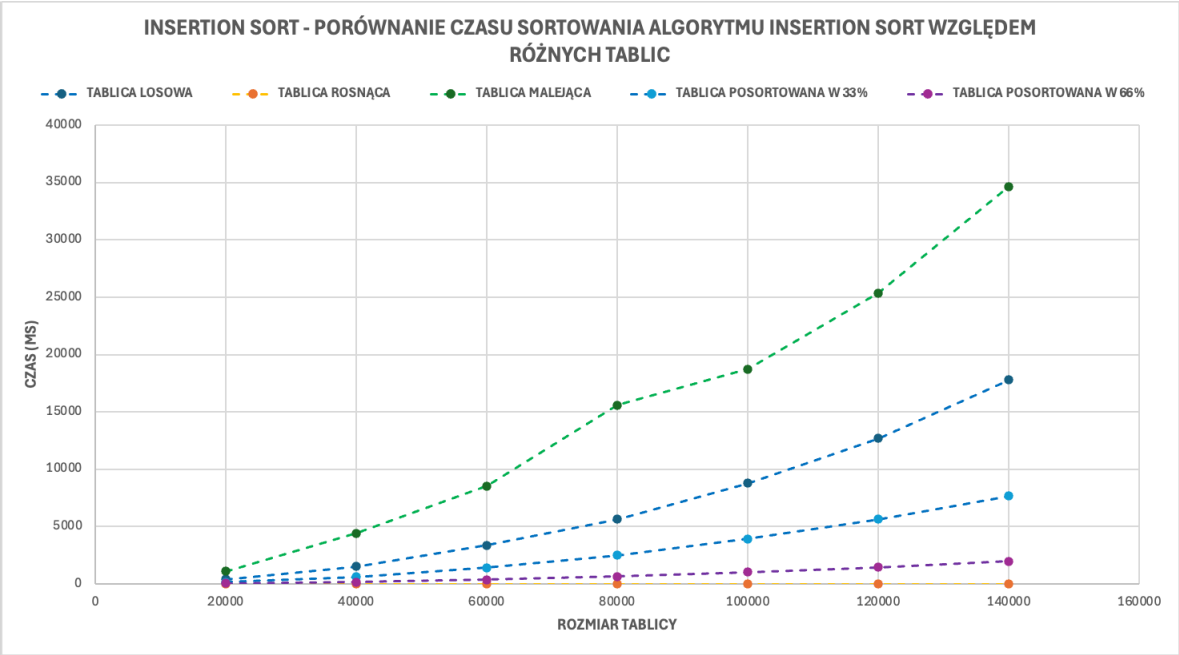
Wykres 4: Wykres czasu sortowania Insertion Sort w funkcji rozmiaru tablicy - tablica posortowana w 66%

2.2.5 Insertion sort dla tablicy posortowanej w 33%



Wykres 5: Wykres czasu sortowania Insertion Sort w funkcji rozmiaru tablicy - tablica posortowana w 33%

2.2.6 Porównanie algorytmu insertion sort w zależności od tablicy początkowej



Wykres 6: Wykres porównujący średnią prędkość sortowania Insertion Sort dla różnych tablic początkowych

2.3 Wnioski

Czas sortowania tablic przez algorytm Insertion Sort charakteryzuje się drastycznnym wzrostem czasu wykonania w miarę wzrostu rozmiaru tablicy do posortowania. Jest to zgodne z teoretyczną złożonością obliczeniową wynoszącą $O(n^2)$. Wartym odnotowania jest fakt, że algorytm wykonuje się w bardzo szybkim tępie dla tablic już na wstępie posortowanych rosnąco - np. średni czas sortowania dla tablicy o rozmiarze 140 tysięcy elementów wyniósł zaledwie 0.29ms - znacznie mniej niż w przypadku pozostałych badanych algorytmów. Insertion Sort może znaleźć zastosowanie w przypadkach, gdzie dane są wstępnie w dużej mierze uporządkowane. Dla nieuporządkowanych elementów, zwłaszcza gdy jest ich dużo, algorytm ten okazuje się wysoce niewydajny.

3 Sortowanie Shell'a

3.1 Wprowadzenie

Sortowanie Shella (Shell sort) to algorytm zbliżony działaniem do algorytmu Insertion Sort. Jest znacznie bardziej efektywny od Insertion Sorta dla większych i mniej uporządkowanych zbiorów elementów. Algorytm ten nie jesst algorytmem stabilnym, oznacza to, że może zmieniać elementy mające tą samą wartość. Działa w miejscu (in-place) - nie wymaga dodatkowej pamięci. Główną ideą Shell Sorta jest porównywanie elementów tablicy znajdujących się w pewnych odstępach od siebie zamiast bezpośrednio sąsiadujących. Na początku sortuje on elementy tablicy położone daleko od siebie, stopniowo zmniejszając odstępy między sortowanymi elementami, aż w końcu wykonuje się zwykłe sortowanie przez wstawianie na wstępnie uporządkowanej tablicy. [6] [3]

```
n := length(A)
gap := n / 2
while gap > 0
    for i := gap to n - 1
        temp := A[i]
        j := i
        while j >= gap and A[j - gap] > temp
            A[j] := A[j - gap]
            j := j - gap
        end while
        A[j] := temp
    end for
    gap := gap / 2
end while
```

Tekst 2: Shell sort (klasyczny) pseudokod

Odległości w jakich znajdują się od siebie porównywane dane są określone przez dobrany wzór. Istnieje wiele wariantów algorytmu Shella bazujących na innych wzorach wyznaczających sekwencję odległości. Dobór wzoru ma zasadniczy wpływ na złożoność czasową tego algorytmu. W tym badaniu zmierzono prędkość sortowania dwóch wariantów algorytmu korzystających z następujących wyrazów ogólnych ciągów:

- klasyczna sekwencja Shell – $\left\lfloor \frac{N}{2^k} \right\rfloor$
- Knuth – $\frac{3^k-1}{2}$

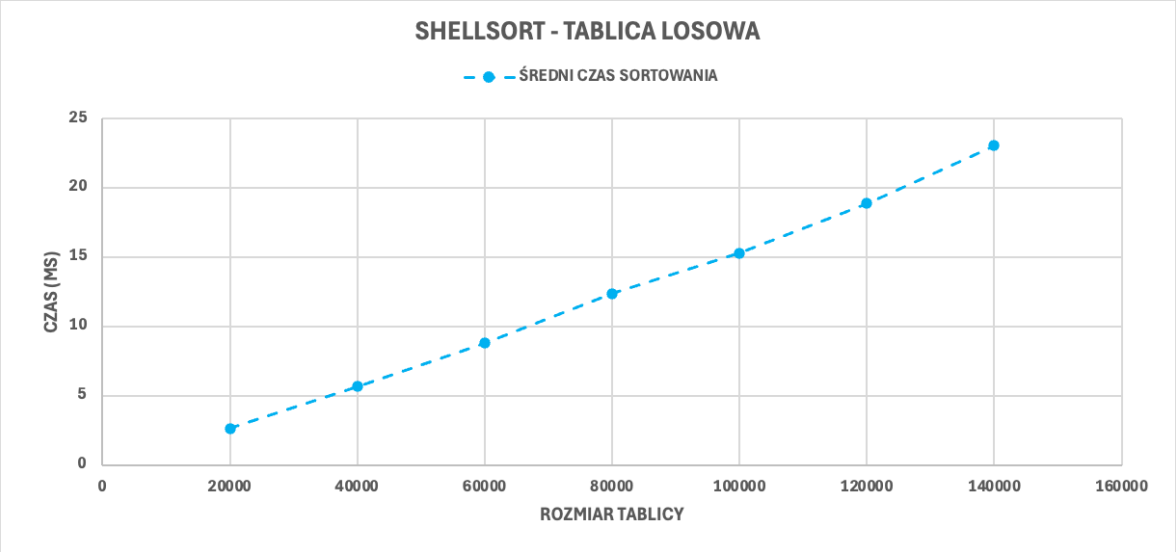
Złożoność czasowa Shell Sorta może znacznie się różnić w zależności od zastosowanego wzoru na sekwencję odstępów. W najgorszym przypadku Shell Sort osiąga złożoność $O(n^2)$, jednak dla np. sekwencji Knutha jest to złożoność rzędu $O(n^{3/2})$. W poniższej tabeli przedstawione zostały najlepsze i najgorsze przypadki dla badanych w tym eksperymencie wariantów algorytmu.

Tabela 2: Porównanie złożoności obliczeniowej ShellSort

Sekwencja odstępów	**Najlepszy przypadek**	**Najgorszy przypadek**
Shell	$O(n \log n)$	$O(n^2)$
Knuth	$O(n \log n)$	$O(n^{3/2})$

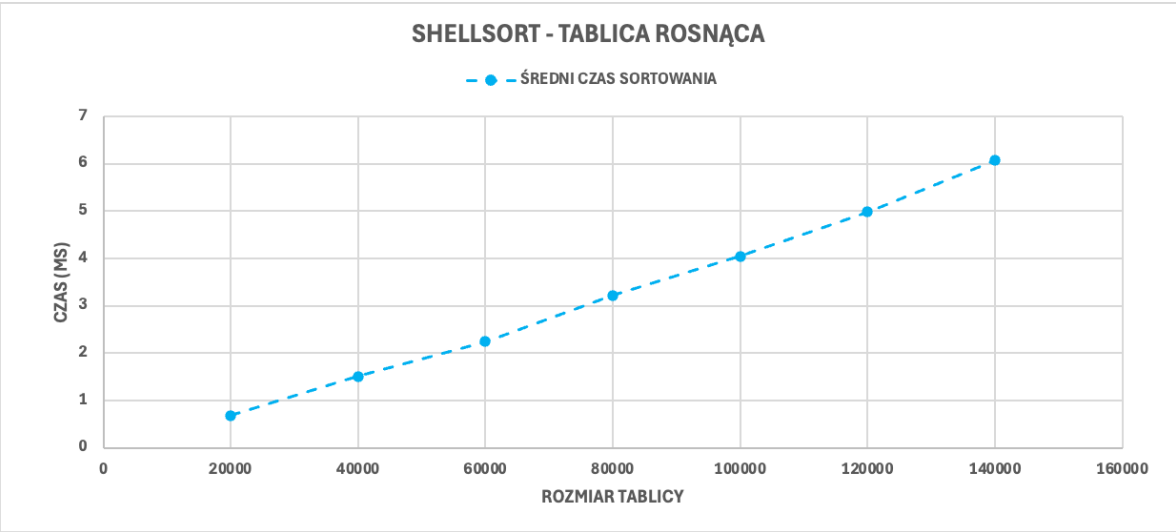
3.2 Wykresy przedstawiające uzyskane wyniki

3.2.1 Shellsort - tablica losowa



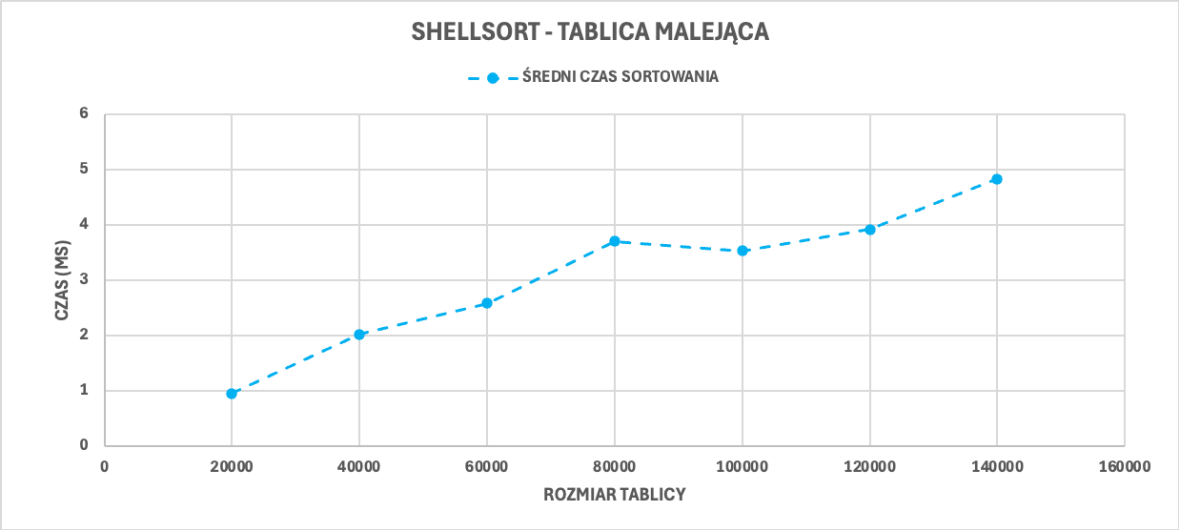
Wykres 7: Wykres czasu sortowania ShellSort w funkcji rozmiaru tablicy - tablica losowa

3.2.2 Shellsort - tablica posortowana rosnąco



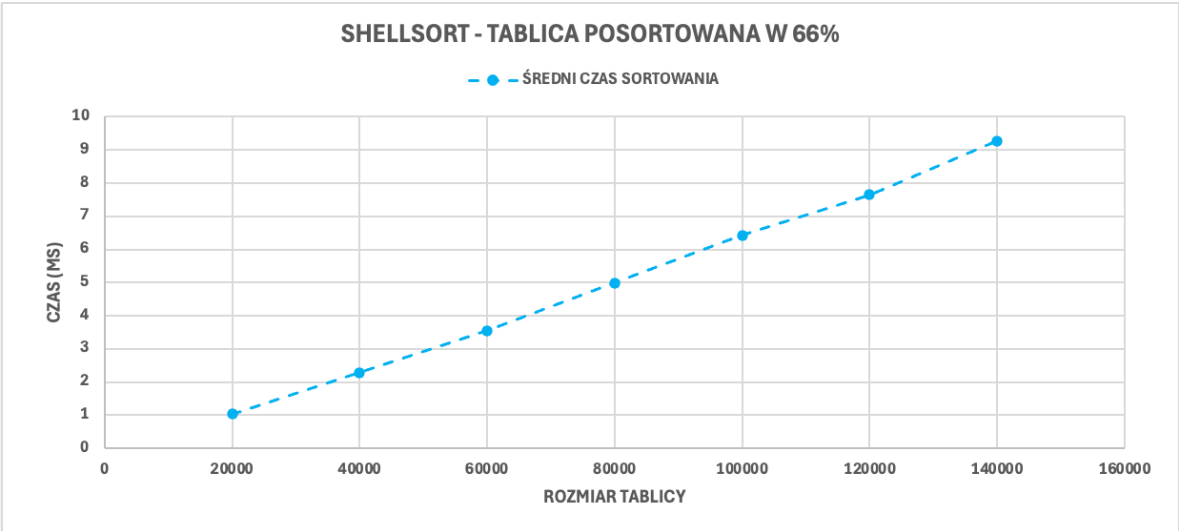
Wykres 8: Wykres czasu sortowania ShellSort w funkcji rozmiaru tablicy - tablica rosnąca

3.2.3 Shellsort - tablica posortowana malejąco



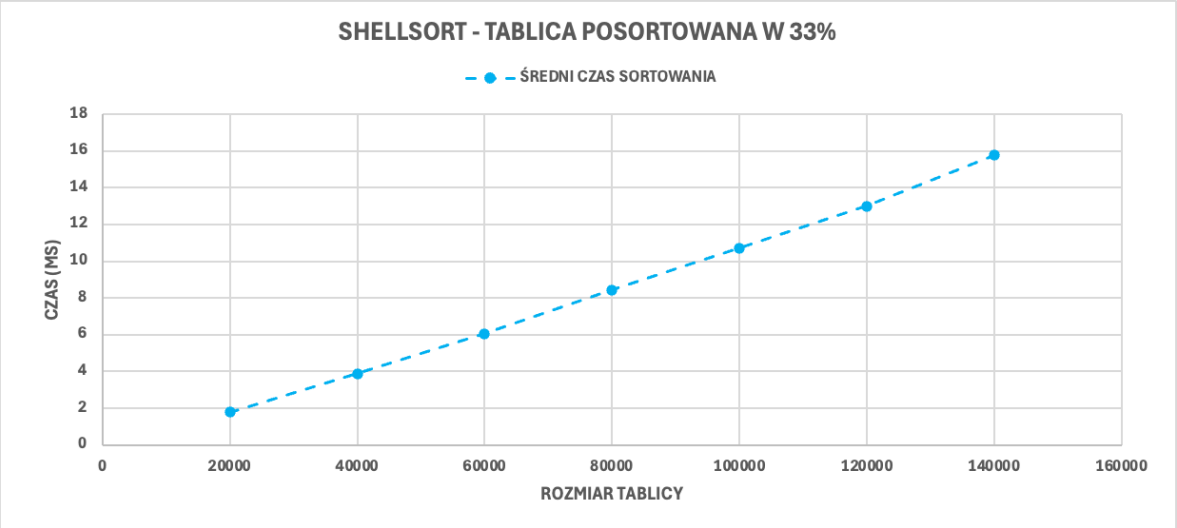
Wykres 9: Wykres czasu sortowania ShellSort w funkcji rozmiaru tablicy - tablica malejąca

3.2.4 Shellsort - tablica posortowana w 66%



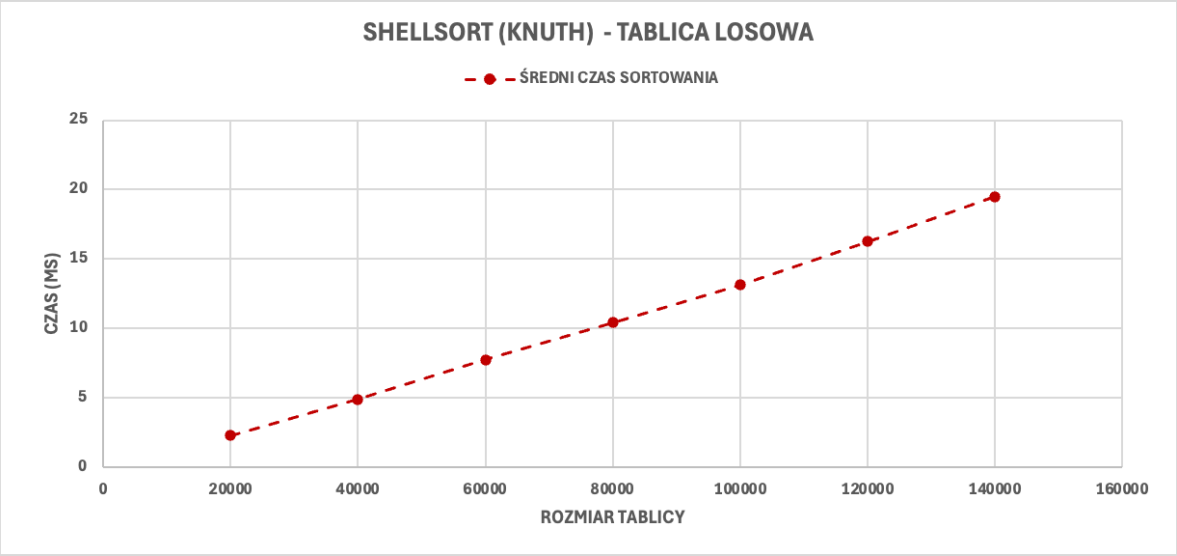
Wykres 10: Wykres czasu sortowania ShellSort w funkcji rozmiaru tablicy - tablica posortowana w 66%

3.2.5 Shellsort - tablica posortowana w 33%



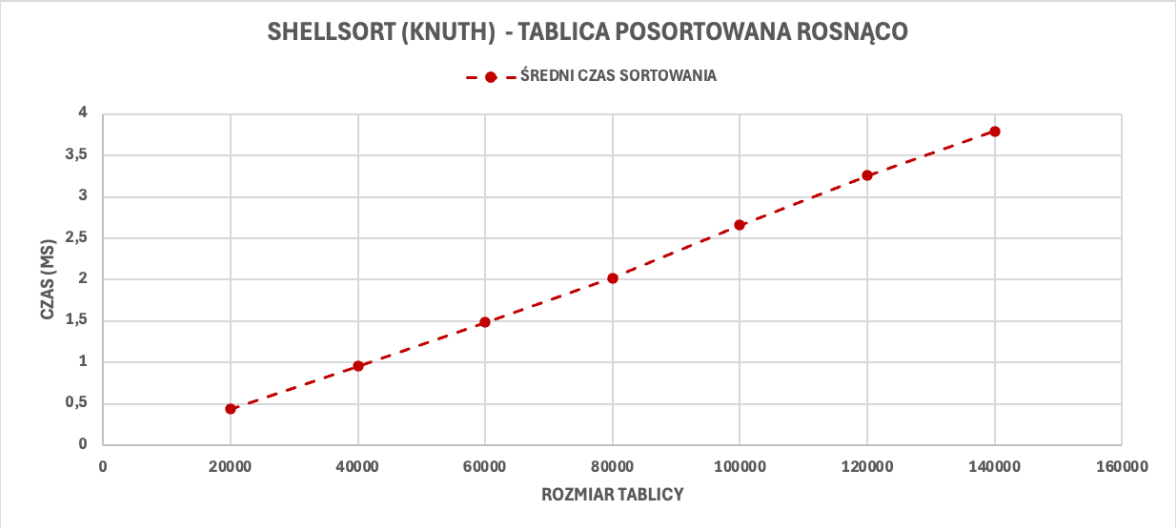
Wykres 11: Wykres czasu sortowania ShellSort w funkcji rozmiaru tablicy - tablica posortowana w 33%

3.2.6 Shellsort(knuth) - tablica losowa



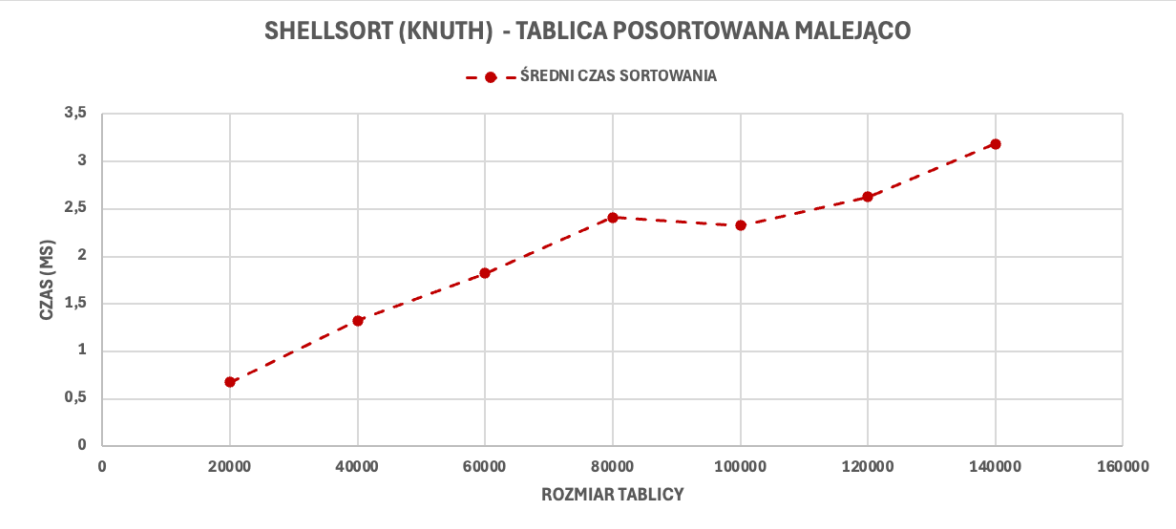
Wykres 12: Wykres czasu sortowania ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica losowa

3.2.7 Shellsort(knuth) - tablica posortowana rosnąco



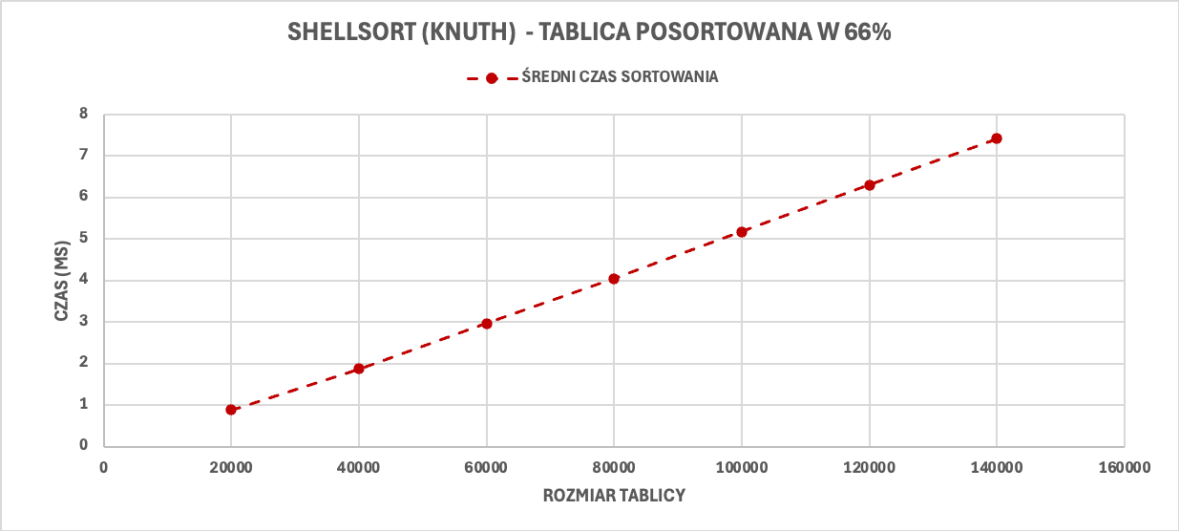
Wykres 13: Wykres czasu sortowania ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica rosnąca

3.2.8 Shellsort(knuth) - tablica posortowana malejąco



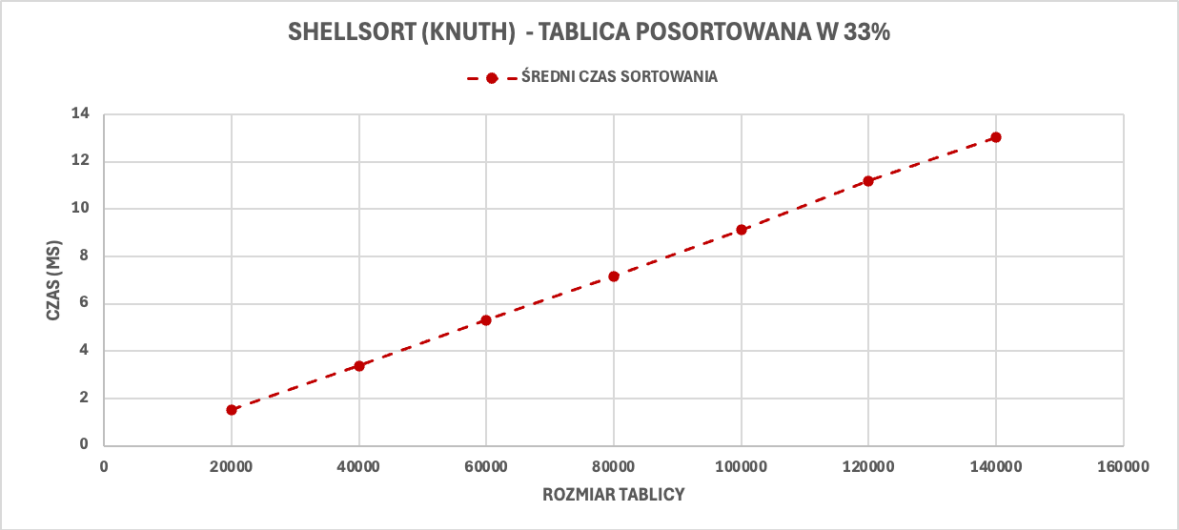
Wykres 14: Wykres czasu sortowania ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica malejąca

3.2.9 Shellsort(knuth) - tablica posortowana w 66%



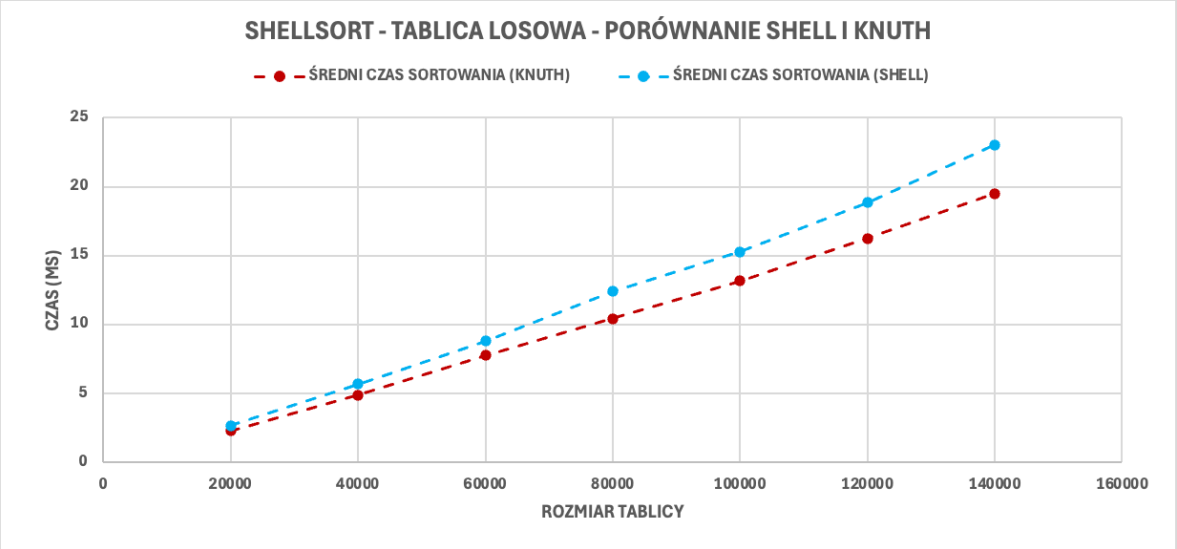
Wykres 15: Wykres czasu sortowania ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica posortowana w 66%

3.2.10 Shellsort(knuth) - tablica posortowana w 33%



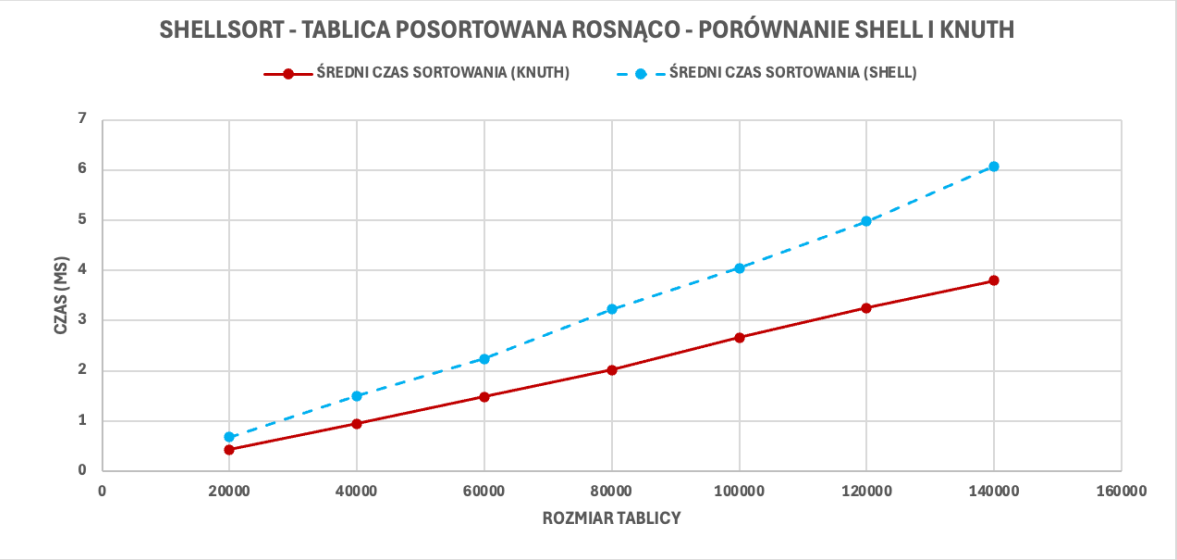
Wykres 16: Wykres czasu sortowania ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica posortowana w 33%

3.2.11 Porównanie dla tablicy losowej



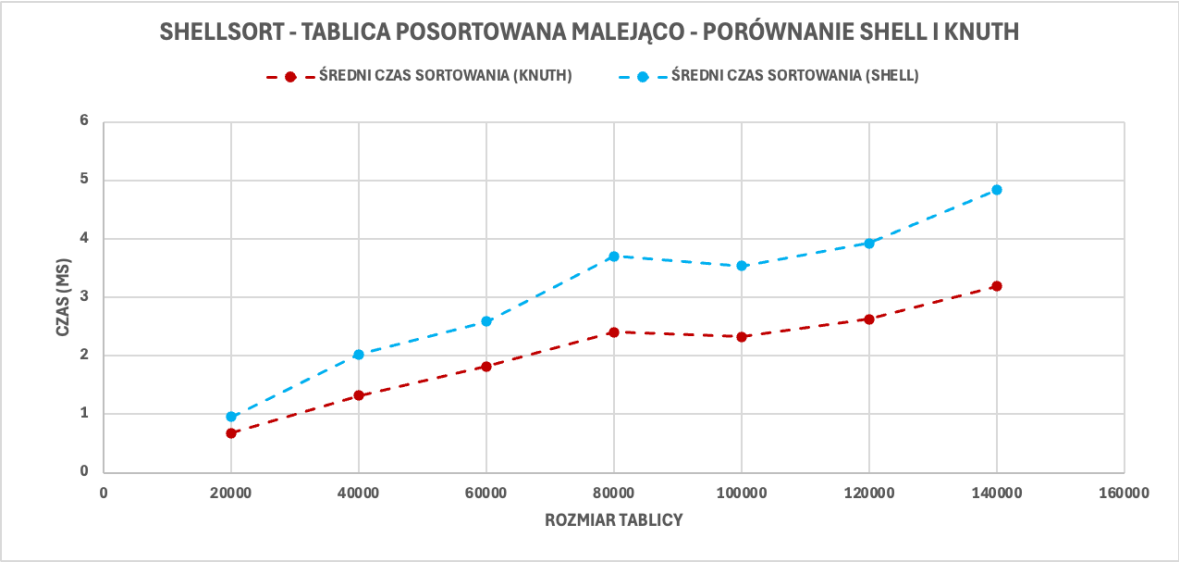
Wykres 17: Porównanie czasu sortowania ShellSort(Shell) i ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica losowa

3.2.12 Porównanie dla tablicy posortowanej rosnąco



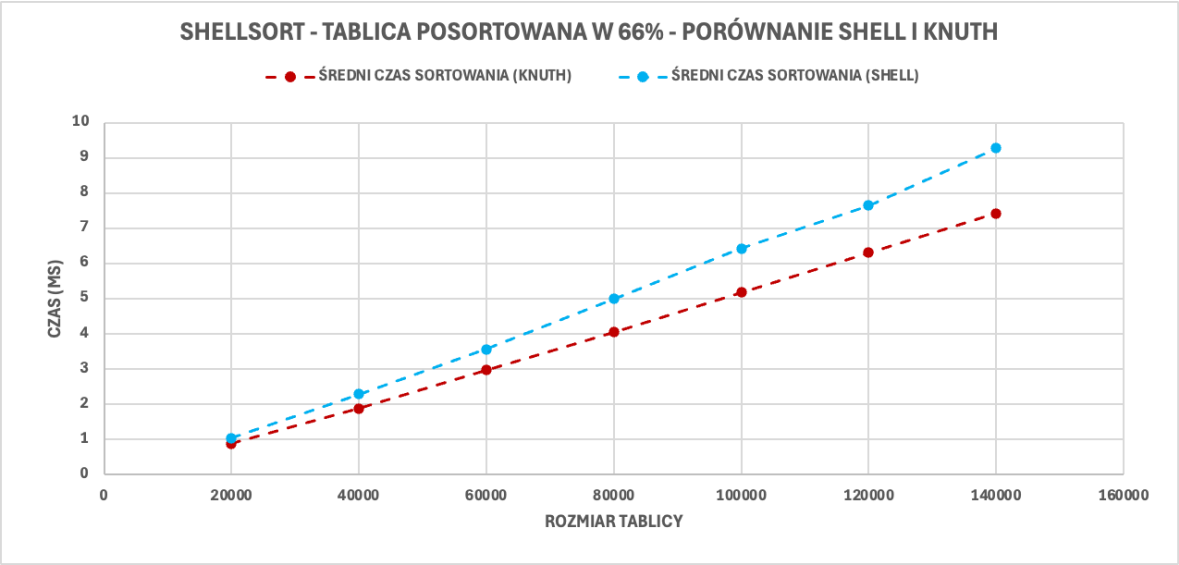
Wykres 18: Porównanie czasu sortowania ShellSort(Shell) i ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica rosnąca

3.2.13 Porównanie dla tablicy posortowanej malejąco



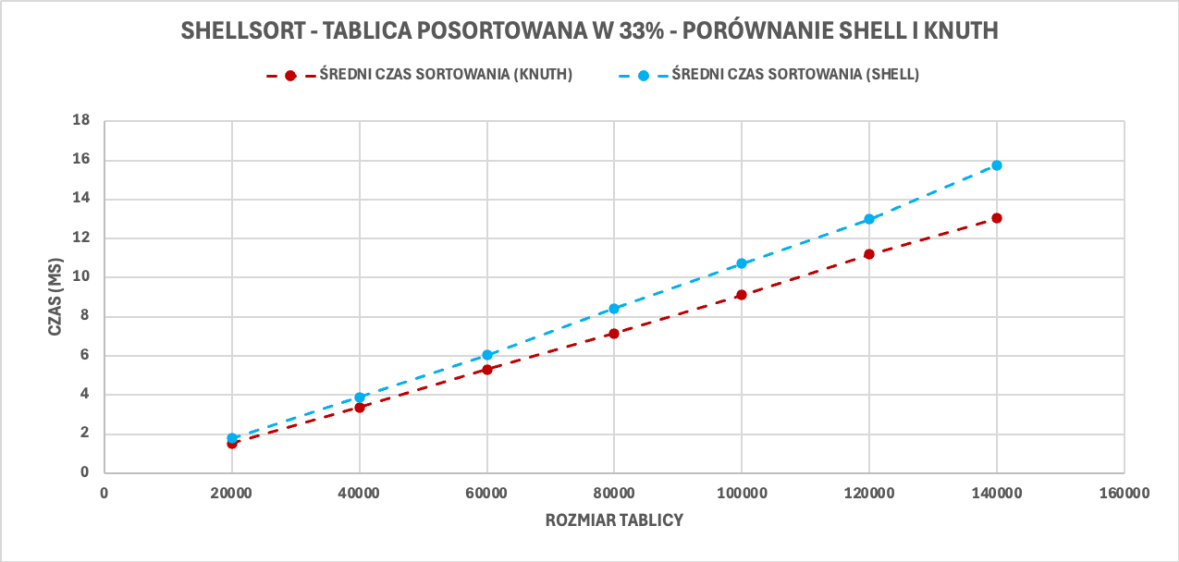
Wykres 19: Porównanie czasu sortowania ShellSort(Shell) i ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica malejąca

3.2.14 Porównanie dla tablicy posortowanej w 66%



Wykres 20: Porównanie czasu sortowania ShellSort(Shell) i ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica posortowana w 66%

3.2.15 Porównanie dla tablicy posortowanej w 33%



Wykres 21: Porównanie czasu sortowania ShellSort(Shell) i ShellSort(Knuth) w funkcji rozmiaru tablicy - tablica posortowana w 33%

3.3 Wnioski

Wyniki eksperymentalnie potwierdzają teoretyczną złożoność czasową. Algorytm Shella osiąga zdecydowanie lepsze wyniki niż klasyczny Insertion Sort dla rozważanych w eksperymencie dużych rozmiarów tablic. Analizując wykresy porównujące klasyczny wyraz ogólny ciągu Shella z wariantem Knutha, ten drugi wypadł lepiej w niemal każdym przypadku. Różnica między czasem wykonania algorytmów jest szczególnie zauważalna w tablicach uporządkowanych rosnąco i malejąo. Z przeprowadzonych pomiarów i powyższych wykresów wynika, że dobór wyrazu ogólnego ciągu w algorytmie Shella może mieć znaczący wpływ na wydajność sortowania.

4 Sortowanie szybkie

4.1 Wprowadzenie

Sortowanie szybkie (quicksort) - jeden z najczęściej używanych algorytmów sortowania. Działa na zasadzie "divide and conquer" (dziel i zwyciężaj). Quicksort nie jest algorytmem stabilnym, elementy o jednakowym kluczu mogą zamienić się miejscami, jednak jest algorytmem działającym w miejscu, nie wymaga dodatkowej pamięci. Główną ideą algorytmu quicksort jest podział tablicy na dwie części poprzez wybór elementu rozdzielającego (pivotu). W kolejnych krokach algorytmu do lewej części tablicy trafiają elementy mniejsze lub równe pivotowi, a do prawej części tablicy trafiają elementy większe. Proces ten następnie jest rekurencyjnie powtarzany dla każdej podtablicy. Rekursja kończy się w momencie uzyskania jednoelementowej tablicy która nie wymaga sortowania. [1] [4]

```
QUICKSORT(A, p, r)
    if p < r
        q := PARTITION(A, p, r)
        QUICKSORT(A, p, q)
        QUICKSORT(A, q + 1, r)


PARTITION(A, p, r)
    x := A[p]
    i := p - 1
    j := r + 1
    while TRUE
        repeat
            j := j - 1
        until A[j]     x

        repeat
            i := i + 1
        until A[i]     x

        if i < j
            swap A[i] with A[j]
        else
            return j
```

Tekst 3: Quick Sort pseudokod

Wybór pivota ma bardzo istotny wpływ na czas sortowania przez algorytm. W projekcie przetestowano różne warianty wyboru pivota:

- pivot środkowy
- pivot wybrany losowo
- pivot skrajnie lewy
- pivot skrajnie prawy

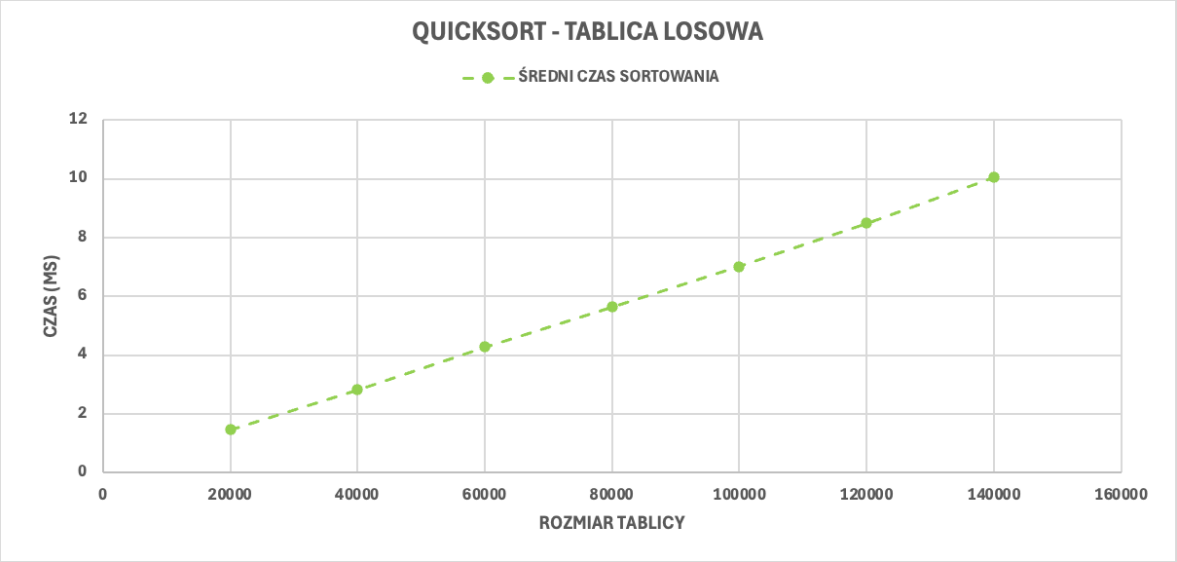
Złożoność czasowa algorytmu quicksort zależy w dużej mierze od wyboru pivota. W najlepszym przypadku, czyli wtedy kiedy pivot dzieli zawsze tablicę na dwie równe części, algorytm osiąga złożoność czasową na poziomie $O(n \log n)$. W przypadku najgorszym, gdy podziały są nierówne np. pivot jest największym elementem, złożoność czasowa wynosi $O(n^2)$. Złożoność pamięciowa wynika z rekursji i wynosi $O(\log n)$

Tabela 3: Złożoność algorytmu Quicksort

Przypadek	Złożoność czasowa	Złożoność pamięciowa
Najlepszy	$O(n \log n)$	$O(\log n)$
Średni	$O(n \log n)$	
Najgorszy	$O(n^2)$	

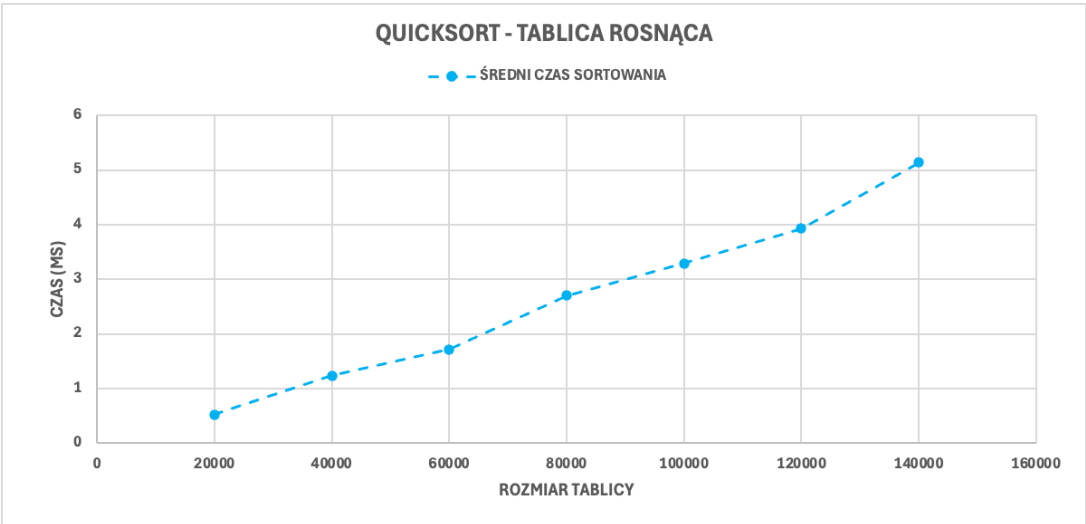
4.2 Wykresy przedstawiające uzyskane wyniki

4.2.1 Quicksort dla tablicy wypełnionej losowymi liczbami



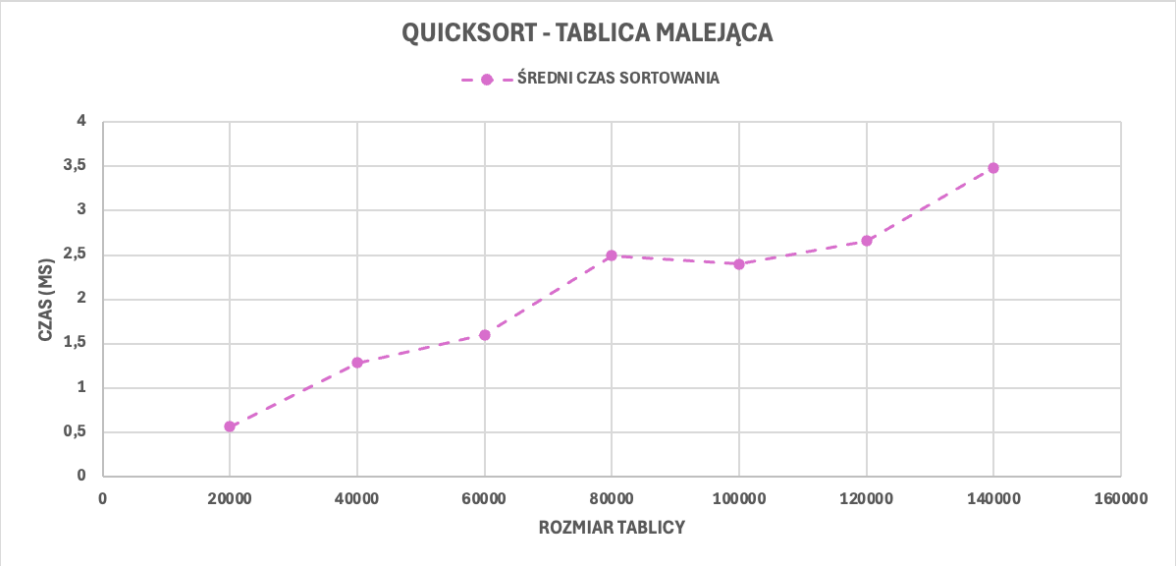
Wykres 22: Wykres czasu sortowania Quick Sort w funkcji rozmiaru tablicy - tablica losowa

4.2.2 Quicksort dla tablicy posortowanej rosnąco



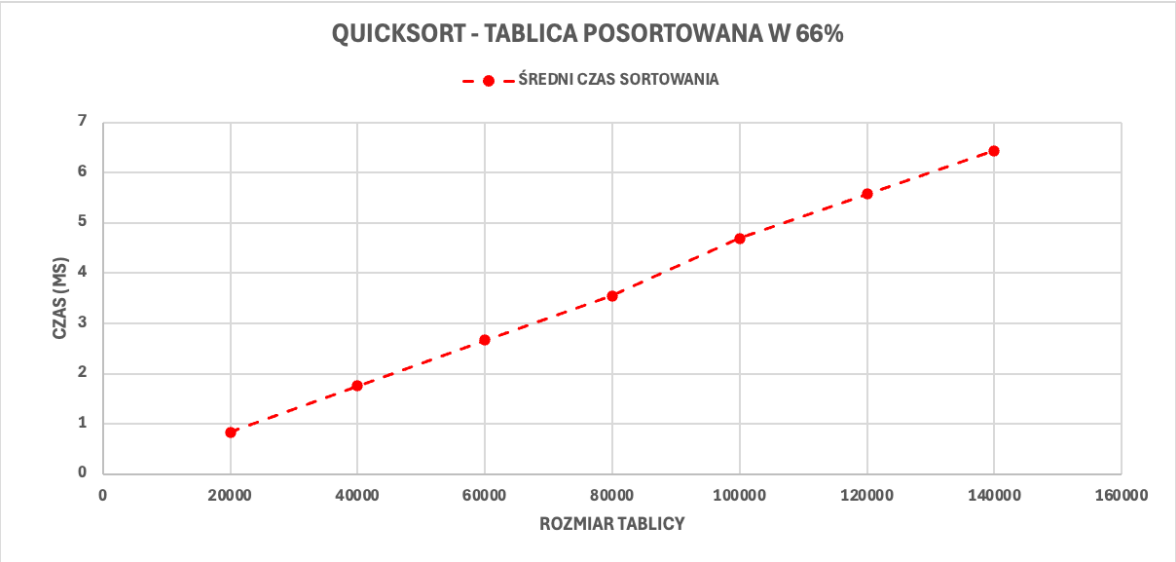
Wykres 23: Wykres czasu sortowania Quick Sort w funkcji rozmiaru tablicy - tablica rosnąca

4.2.3 Quicksort dla tablicy posortowanej malejąco



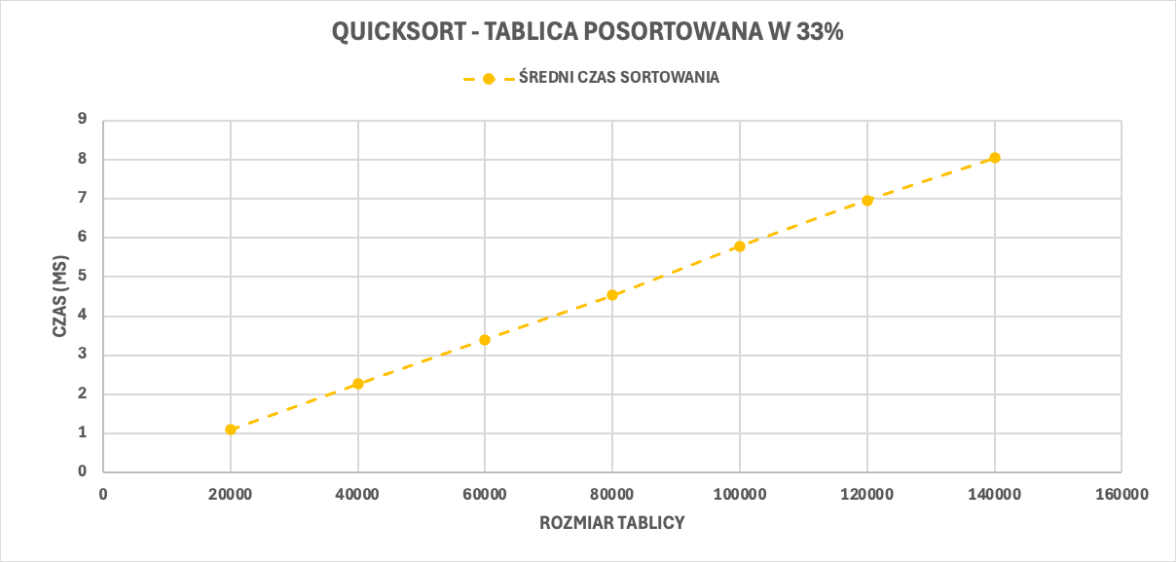
Wykres 24: Wykres czasu sortowania Quick Sort w funkcji rozmiaru tablicy - tablica malejąca

4.2.4 Quicksort dla tablicy posortowanej w 66%



Wykres 25: Wykres czasu sortowania Quick Sort w funkcji rozmiaru tablicy - tablica posortowana w 66%

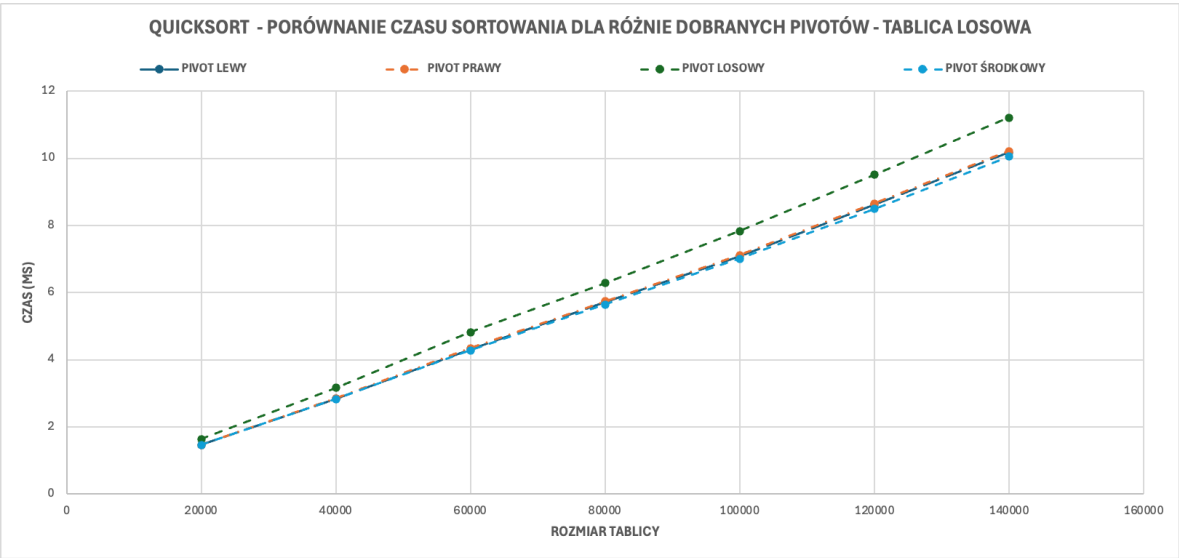
4.2.5 Quicksort dla tablicy posortowanej w 33%



Wykres 26: Wykres czasu sortowania Quick Sort w funkcji rozmiaru tablicy - tablica posortowana w 33%

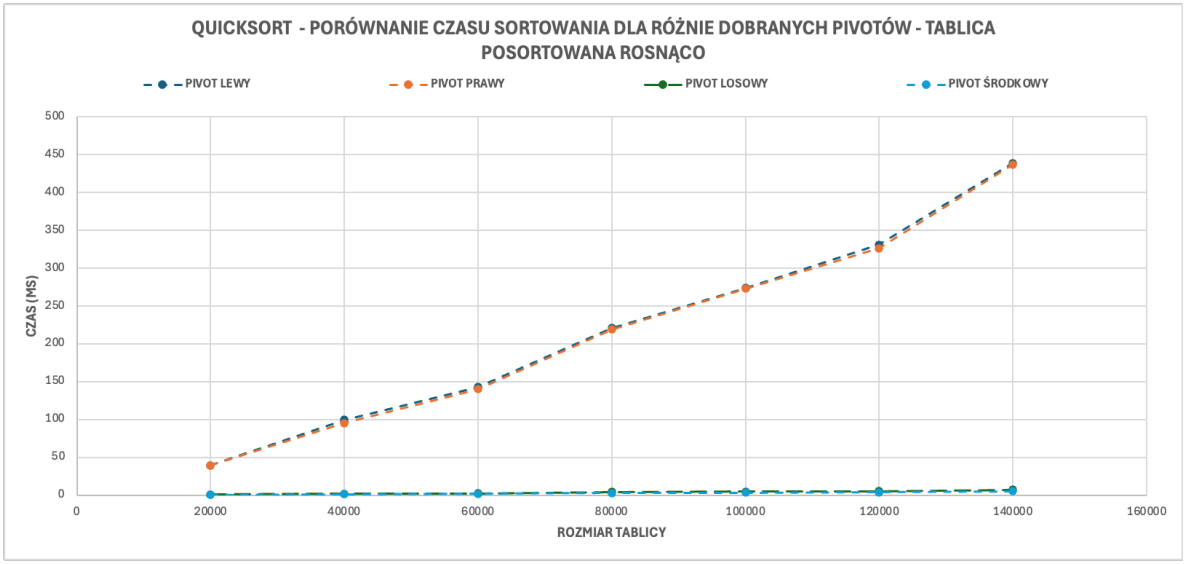
4.3 Porównanie pivotów w algorytmie quicksort oraz wykresy

4.3.1 Dla tablicy losowej



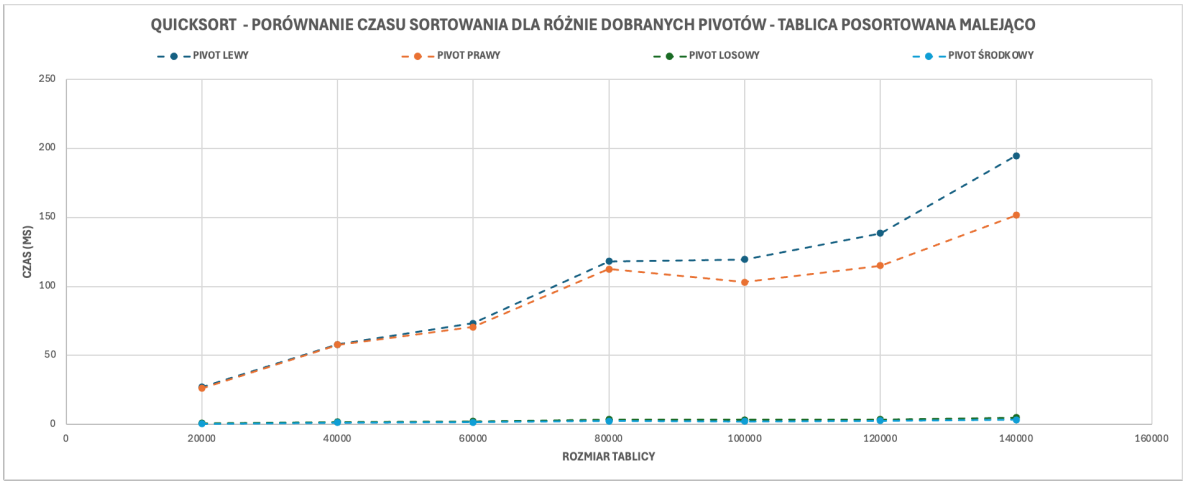
Wykres 27: Porównanie doboru pivotów w Quick Sort - tablica losowa

4.3.2 Dla tablicy posortowanej rosnąco



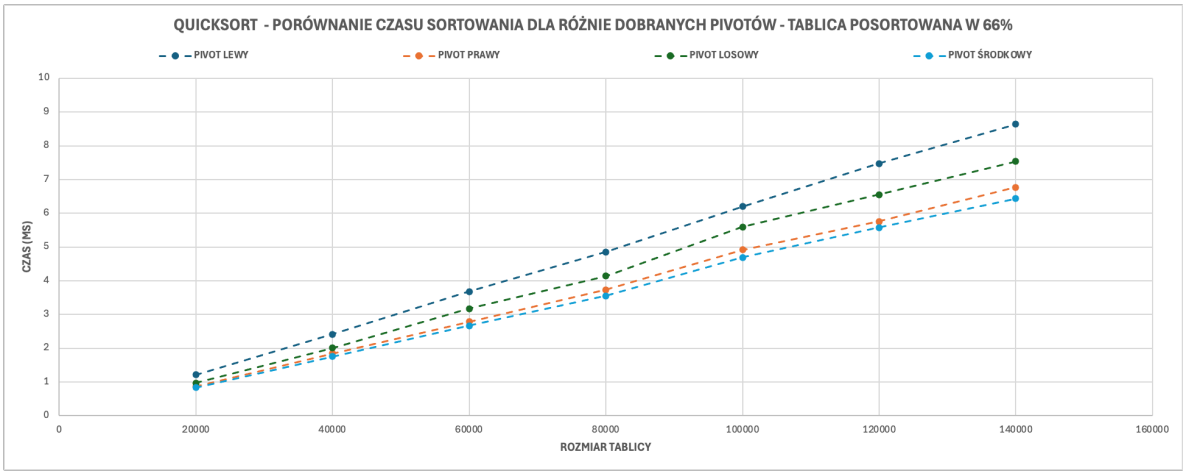
Wykres 28: Porównanie doboru pivotów w Quick Sort - tablica rosnąca

4.3.3 Dla tablicy posortowanej malejąco



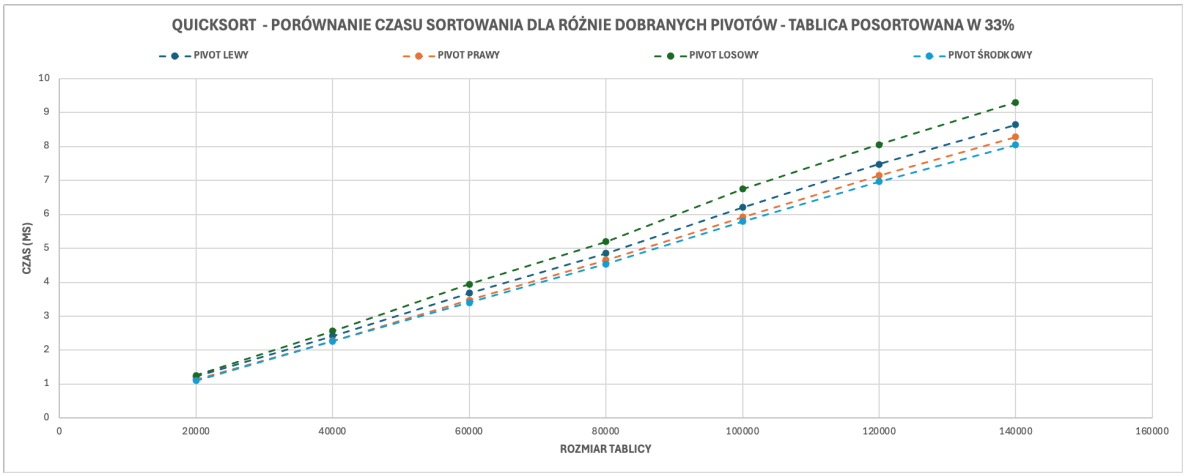
Wykres 29: Porównanie doboru pivotów w Quick Sort - tablica malejąca

4.3.4 Dla tablicy posortowanej w 66%



Wykres 30: Porównanie doboru pivotów w Quick Sort - tablica posortowana w 66%

4.3.5 Dla tablicy posortowanej w 33%



Wykres 31: Porównanie doboru pivotów w Quick Sort - tablica posortowana w 33%

4.4 Wnioski

Średnie wyniki czasu sortowania algorytmu quicksort były zdecydowanie najlepsze z rozważanych w eksperymencie algorytmów sortowania. Algorytm osiągnął czasy sortowania zgodne z przewidywaną złożonością na poziomie $O(n \log n)$. Analizując wykresy są widoczne znaczne spadki wydajności dla tablic uporządkowaych rosnąco lub malejąco w przypadku wyboru pivotów skrajnie lewego lub skrajnie prawego, przypominając przewidywaną pesymistyczną złożoność $O(n^2)$. W odróżnieniu od dwóch skrajnych pivotów znacznie lepsze wyniki uzyskały pivot losowy oraz środkowy. Są one bardziej stabilne i uzyskują najkorzystniejsze czasy sortowania niezależnie od początkowego ułożenia tablicy. Dobór pivota ma kluczowe znaczenie dla wydajności sortowania algorytmu quicksort. Podczas eksperymentalnego doboru rozmiarów tablic do projektu, wykonywanie algorytmu quicksort na

bardzo dużych tablicach (320000 elementów i więcej) kończyło się niepowodzeniem, ponieważ występowały błędy przepełnienia stosu. Problem ten dotyczył głównie wariantów z nieoptymalnym wyborem pivotu (lewego lub prawego). Potwiedza to wyższość wyboru pivotu środkowego lub losowego nad skrajnymi opcjami.

5 Sortowanie przez kopcowanie

5.1 Wprowadzenie

Sortowanie przez kopcowanie (Heap Sort) - algorytm sortowania operujący na strukturze kopca binarnego. Heap sort nie jest algorytmem stabilnym, może zmieniać kolejność tych samych kluczy, działa w miejscu. Algorytm ten polega na tym, że buduje z danych wejściowych kopiec maksymalny aby następnie w kolejnych krokach kasować korzeń kopca (największy element) i umieszczać go na końcu tablicy. Po każdej takiej operacji przywraca wartość kopca (heapify). [5]

```
1 HEAPSORT(A)
2       n := length(A)
3
4       // Budowanie kopca maksymalnego
5       for i := floor(n / 2) - 1 downto 0
6           HEAPIFY(A, n, i)
7
8       // Wyciaganie elementow z kopca
9       for i := n - 1 downto 0
10          swap A[0] with A[i]
11          HEAPIFY(A, i, 0)
12
12 HEAPIFY(A, n, i)
13      largest := i
14      left := 2 * i + 1
15      right := 2 * i + 2
16
17      if left < n and A[left] > A[largest]
18          largest := left
19
20      if right < n and A[right] > A[largest]
21          largest := right
22
23      if largest != i
24          swap A[i] with A[largest]
25          HEAPIFY(A, n, largest)
```

Tekst 4: Heap Sort pseudokod

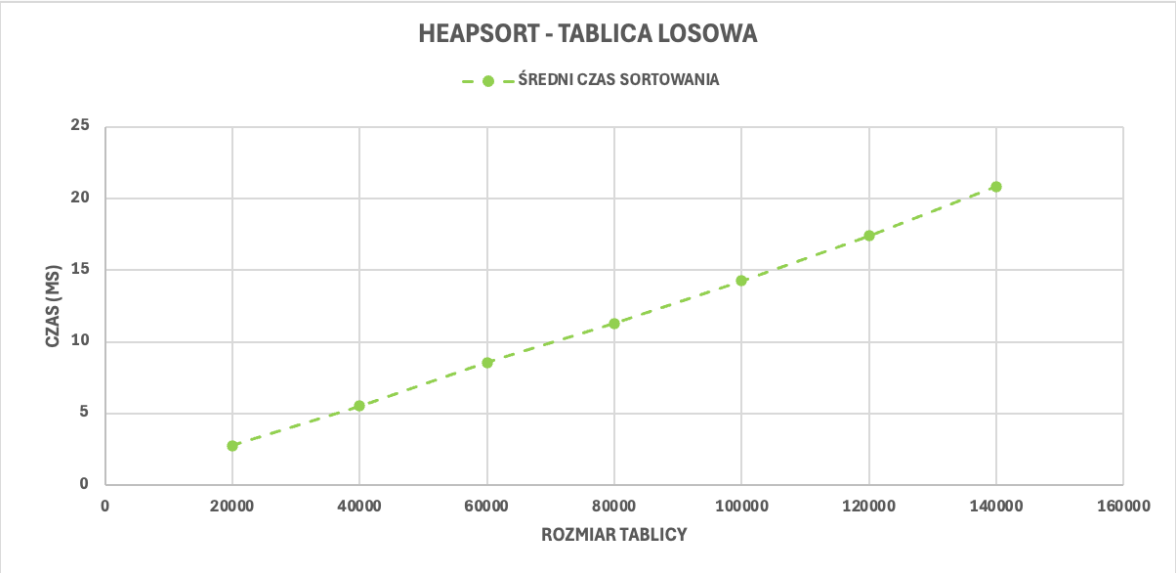
Złożoność obliczeniowa algorytmu HeapSort nie zmienia się niezależnie od struktury danych wejściowych co stanowi zaletę tego algorytmu w stosunku do QuickSorta. Budowanie kopca ma złożoność czasową $O(n)$ a sam proces sortowania, czyli wyciągania kolejnych elementów ze struktury $O(n \log n)$. Łączna złożoność czasowa dla całego procesu wynosi więc $O(n \log n)$ we wszystkich przypadkach

Przypadek	**Złożoność czasowa**	**Złożoność pamięciowa**
Najlepszy	$O(n \log n)$	$O(1)$
Średni	$O(n \log n)$	
Najgorszy	$O(n \log n)$	

Tabela 4: Złożoność algorytmu HeapSort

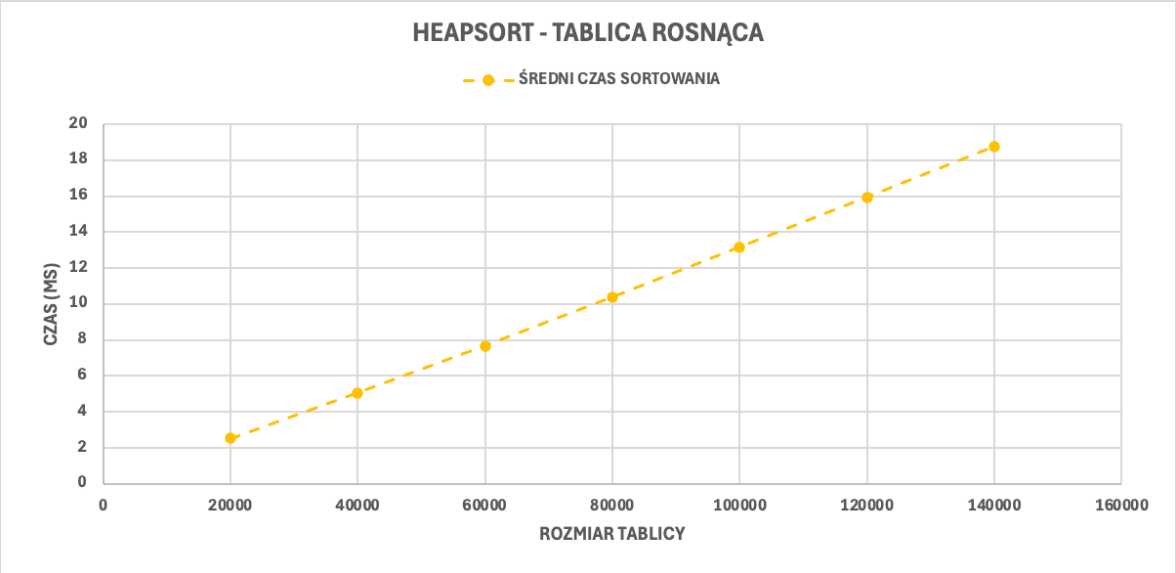
5.2 Wykresy przedstawiające uzyskane wyniki

5.2.1 Heapsort dla tablicy wypełnionej losowymi liczbami



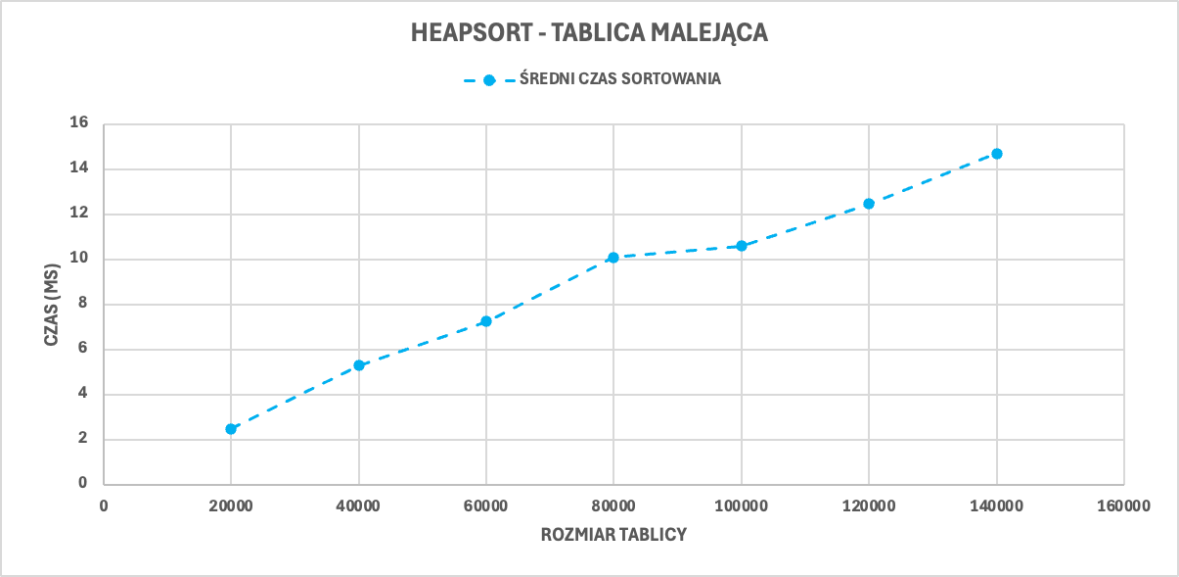
Wykres 32: Wykres czasu sortowania Heapsort w funkcji rozmiaru tablicy - tablica losowa

5.2.2 Heapsort dla tablicy posortowanej rosnąco



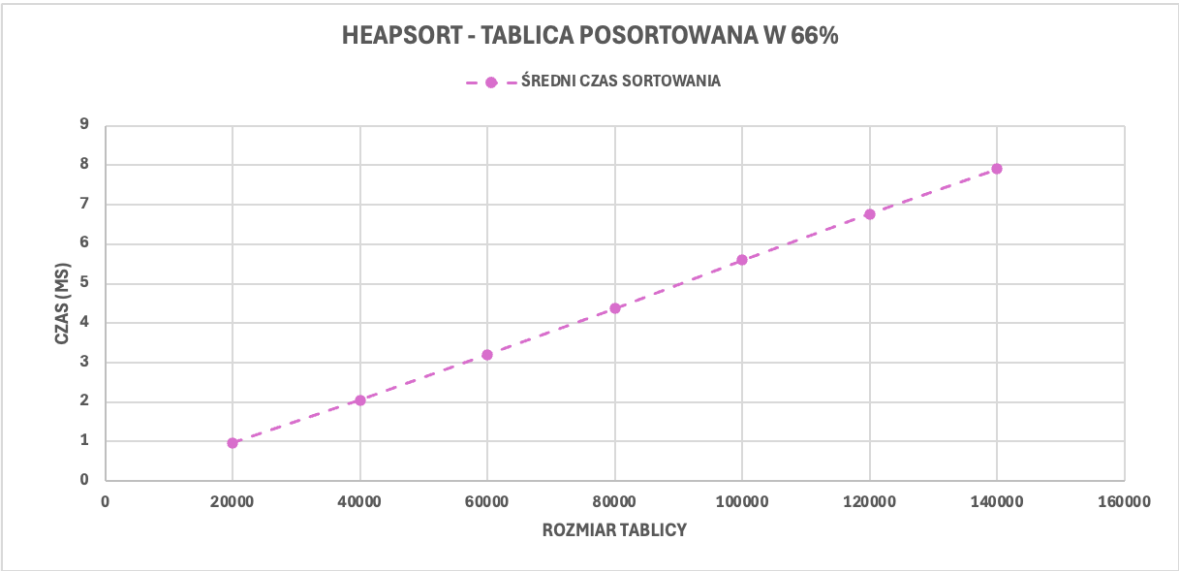
Wykres 33: Wykres czasu sortowania Heapsort w funkcji rozmiaru tablicy - tablica rosnąca

5.2.3 Heapsort dla tablicy posortowanej malejąco



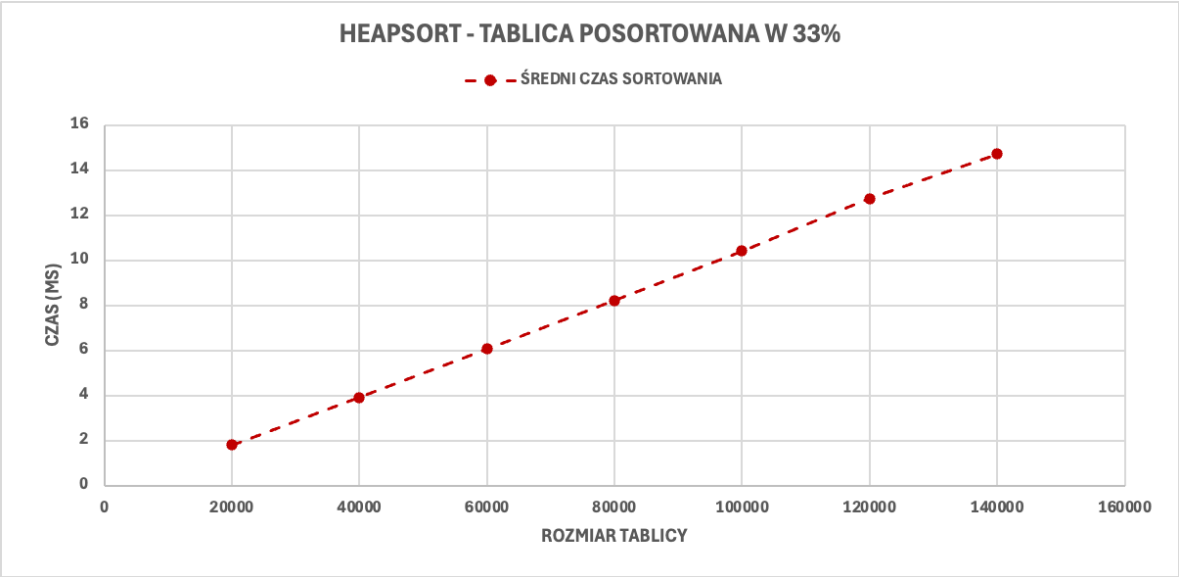
Wykres 34: Wykres czasu sortowania Heapsort w funkcji rozmiaru tablicy - tablica malejąca

5.2.4 Heapsort dla tablicy posortowanej w 66%



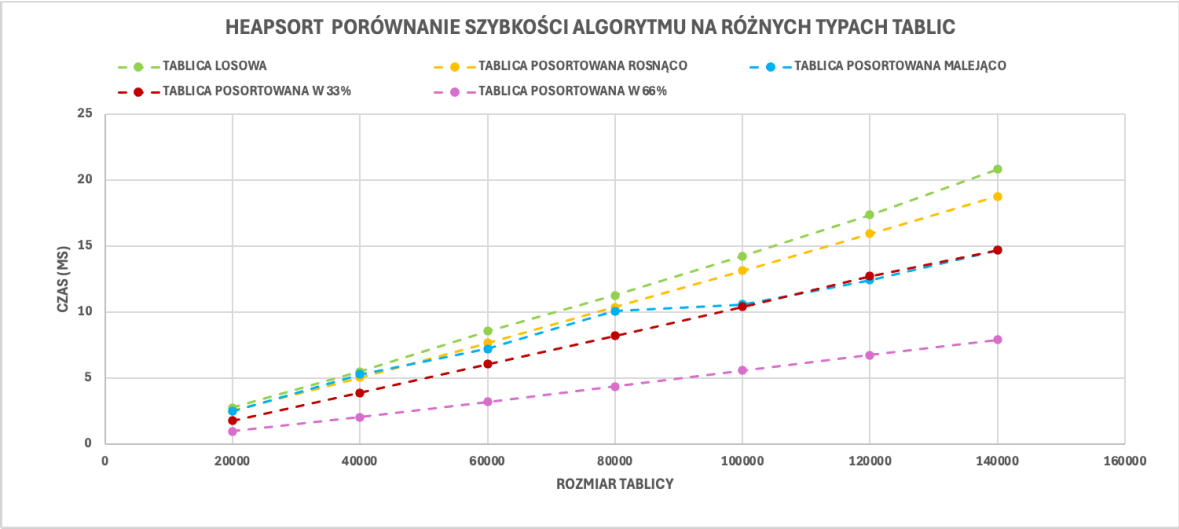
Wykres 35: Wykres czasu sortowania Heapsort w funkcji rozmiaru tablicy - tablica posortowana w 66%

5.2.5 Heapsort dla tablicy posortowanej w 33%



Wykres 36: Wykres czasu sortowania Heapsort w funkcji rozmiaru tablicy - tablica posortowana w 33%

5.2.6 Heapsort - porównanie szybkości algorytmu na różnych typach tablic



Wykres 37: Porównanie szybkości sortowania Heapsort dla poszczególnych rodzajów tablic

5.3 Wnioski

Heapsort zachowuje bardzo zbliżoną, wysoką wydajność niezależnie od stanu początkowego tablicy. Wyniki pokrywają się z teoretyczną złożonością $O(n \log n)$. W porównaniu z innymi algorytmami wyróżnia go przede wszystkim wysoka odporność na zróżnicowane, niekorzystne dane wejściowe. Nie jest to najszybszy algorytm bo pod względem szybkości ustępuje znacznie szybszemu QuickSortowi, jednak znajduje zastosowanie w których kluczowe jest zachowanie wysokiej wydajności niezależnie od skomplikowania struktury danych.

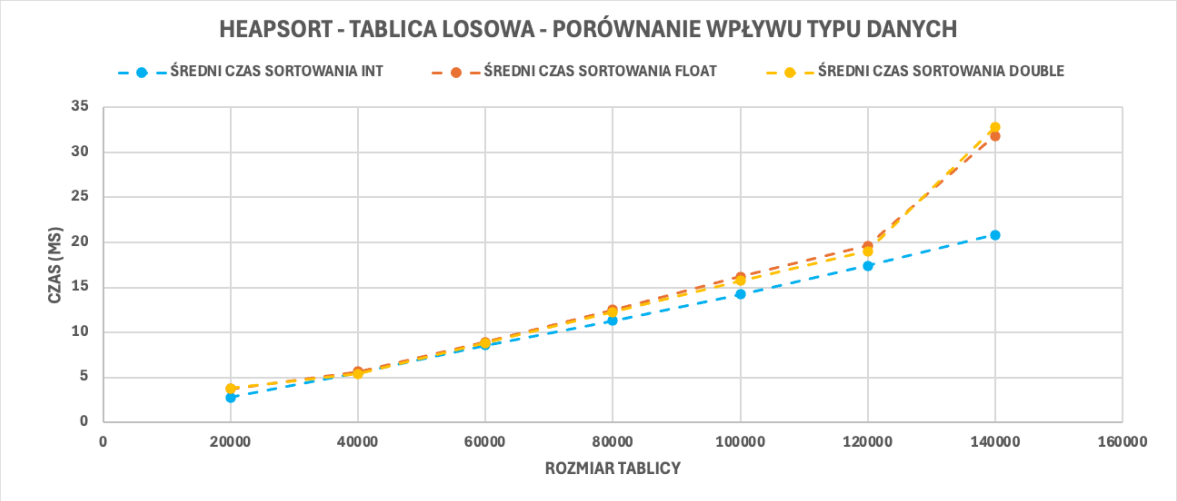
6 Porównanie wpływu typu danych na szybkość sortowania na podstawie sortowania przez kopcowanie

6.1 Wprowadzenie

W ramach projektu przeprowadzono testy wpływu doboru typu danych na szybkość sortowania przez algorytmy sortujące. Testy zostały przeprowadzone na przykładzie algorytmu heap sort, porównano tablice typu int, float oraz double.

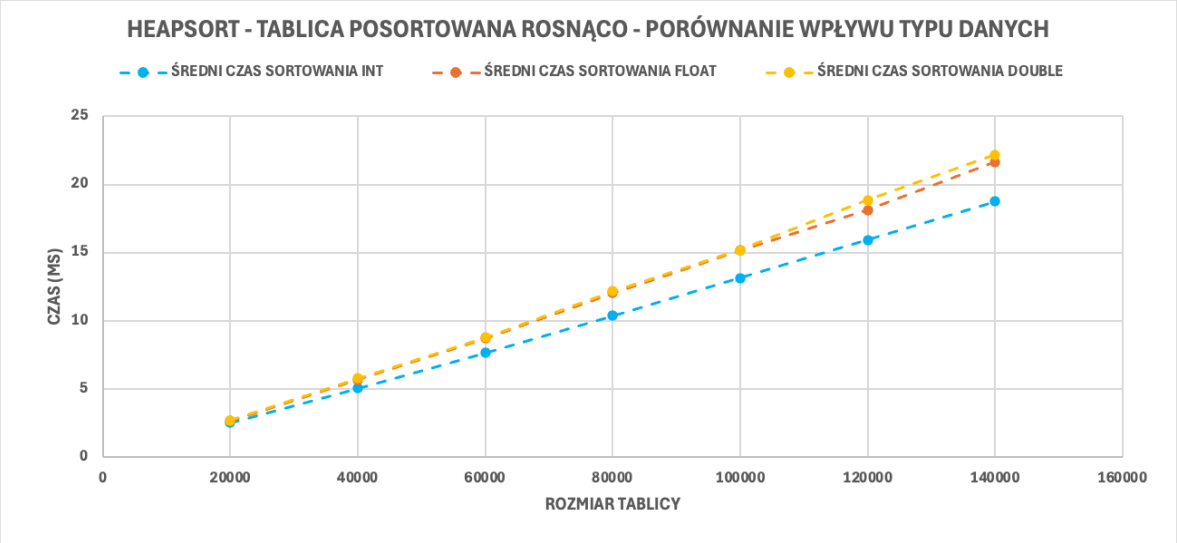
6.2 Wykresy przedstawiające uzyskane wyniki

6.2.1 Dla tablicy losowej



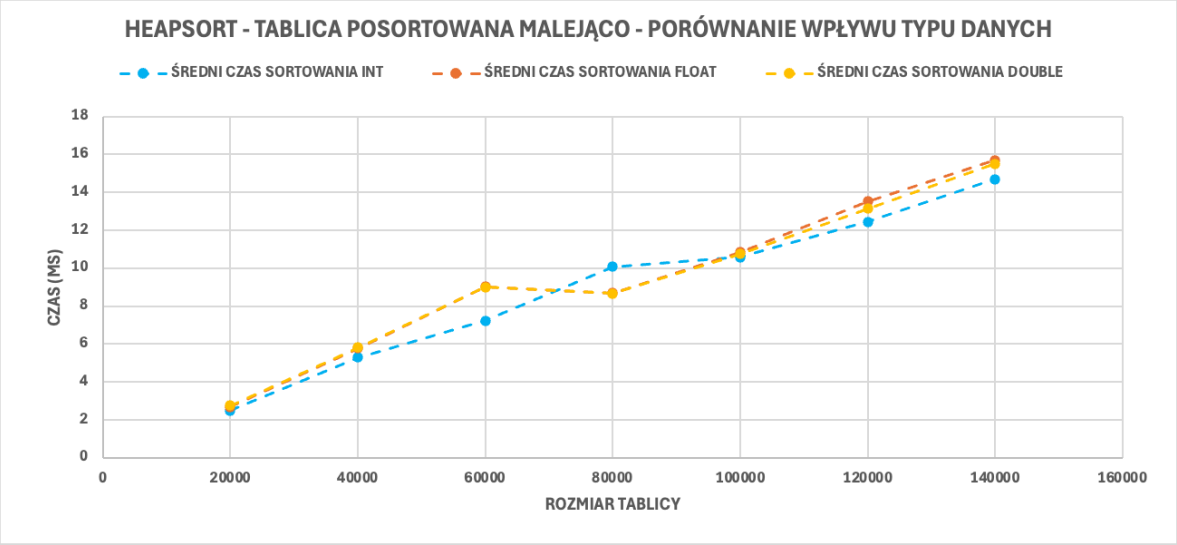
Wykres 38: Wykres porównania typu danych dla tablicy losowej

6.2.2 Dla tablicy posortowanej rosnąco



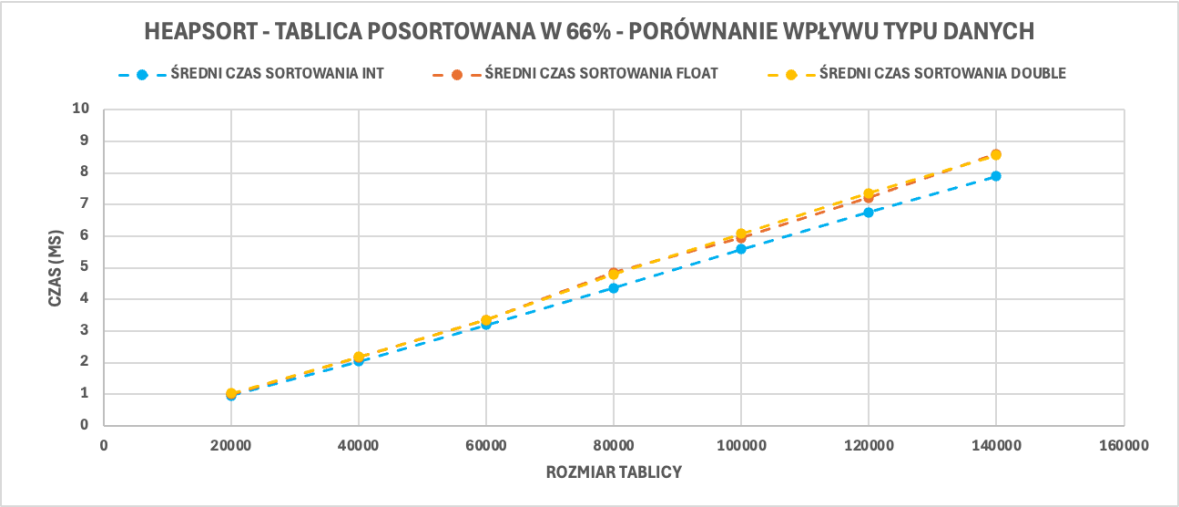
Wykres 39: Wykres porównania typu danych dla tablicy posortowanej rosnąco

6.2.3 Dla tablicy posortowanej malejąco



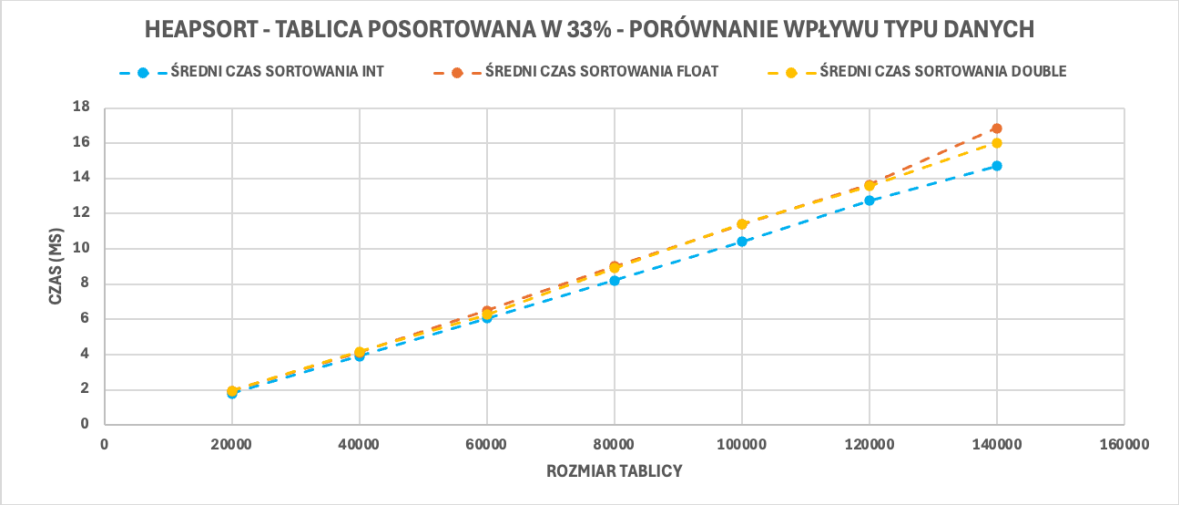
Wykres 40: Wykres porównania typu danych dla tablicy posortowanej malejąco

6.2.4 Dla tablicy posortowanej w 66%



Wykres 41: Wykres porównania typu danych dla tablicy posortowanej w 66%

6.2.5 Dla tablicy posortowanej w 33%



Wykres 42: Wykres porównania typu danych dla tablicy posortowanej w 33%

6.3 Wnioski

Dobór typu danych wpływa nieznacznie na czas sortowania tablicy. Najkrótsze średnie czasy sortowania odnotowano dla typu int. Pozostałe float i double - bardzo zbliżone do siebie wypadły nieco niekorzystniej ze względu na większą precyzję tych typów. Podsumowując dobór typów danych ma niewielki wpływ na czas wykonania sortowania, jednak nie wpływa on na ogólną złożoność czasową algorytmu.

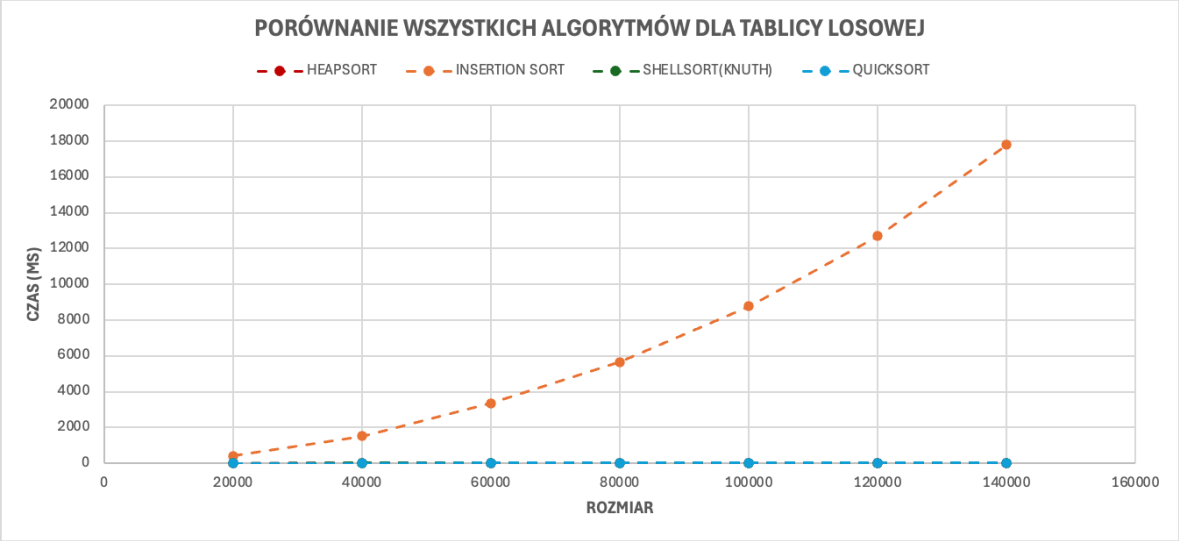
7 Porównanie ogólne wszystkich algorytmów

7.1 Wprowadzenie

W ramach eksperymentu zestawiono również wszystkie rozpatrywane algorytmy na wspólnym wykresie w celu porównania który z algorytmów jest najszybszy. Do porównania zostały wybrane najwydajniesze warianty tj. wariant Knuth dla Shell Sorta oraz QuickSort w wariancie ze środkowym pivotem. Ze względu na bardzo dużą różnicę czasu wykonania Insertion Sort w porównaniu z pozostałymi algorytmami został on umieszczony tylko na pierwszym wykresie aby nie zaburzać skali i poprawić czytelność pozostałych wykresów.

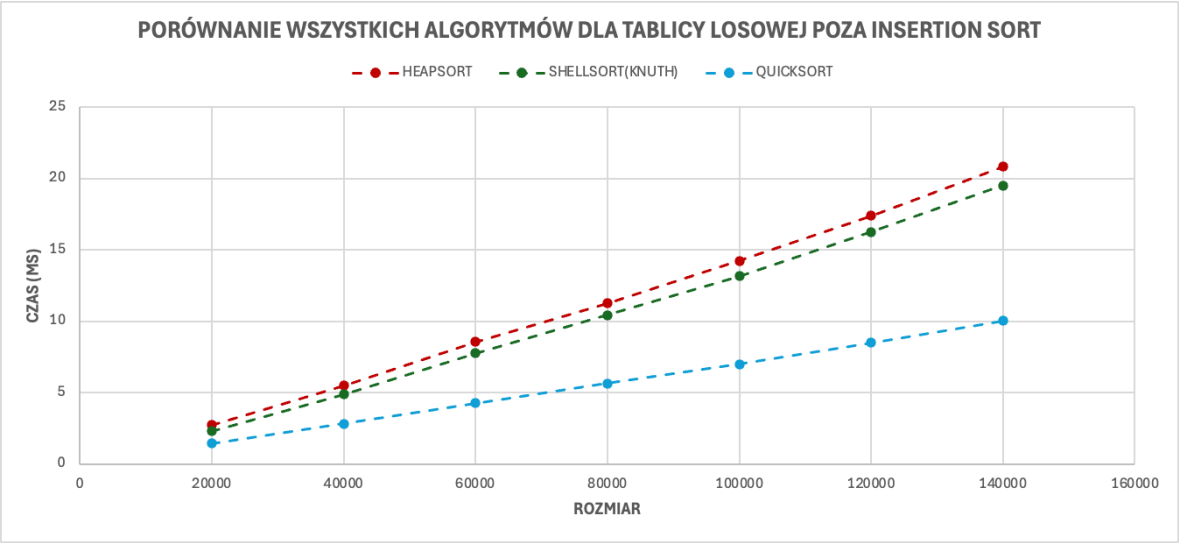
7.2 Wykresy przedstawiające uzyskane wyniki

7.2.1 Porównanie wszystkich przedstawionych w sprawozdaniu algorytmów



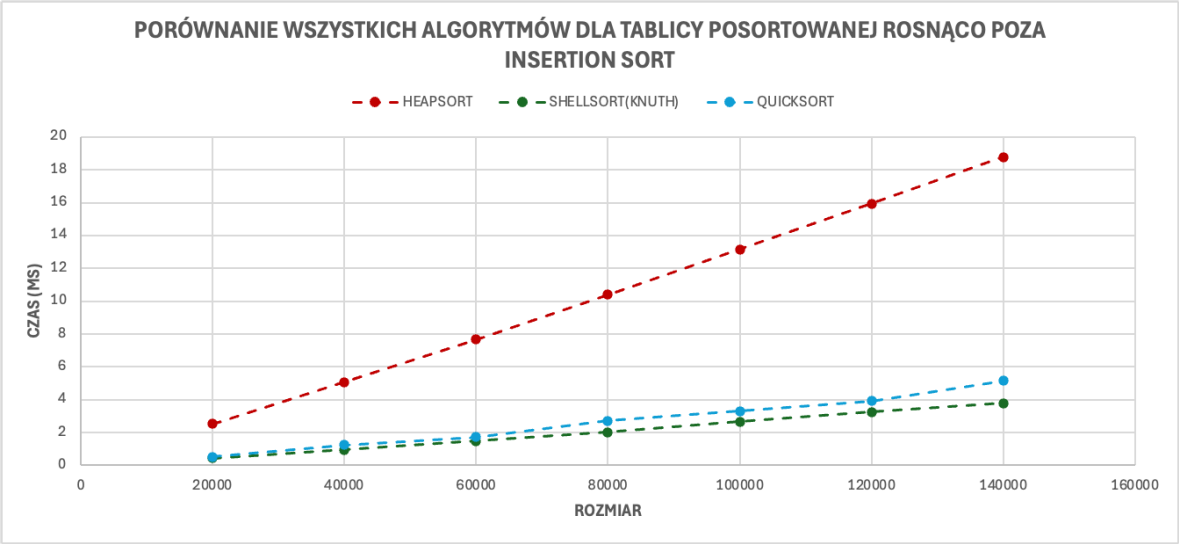
Wykres 43: Wykres czasu sortowania wszystkich algorytmów w funkcji rozmiaru tablicy - tablica losowa

7.2.2 Dla tablicy losowej



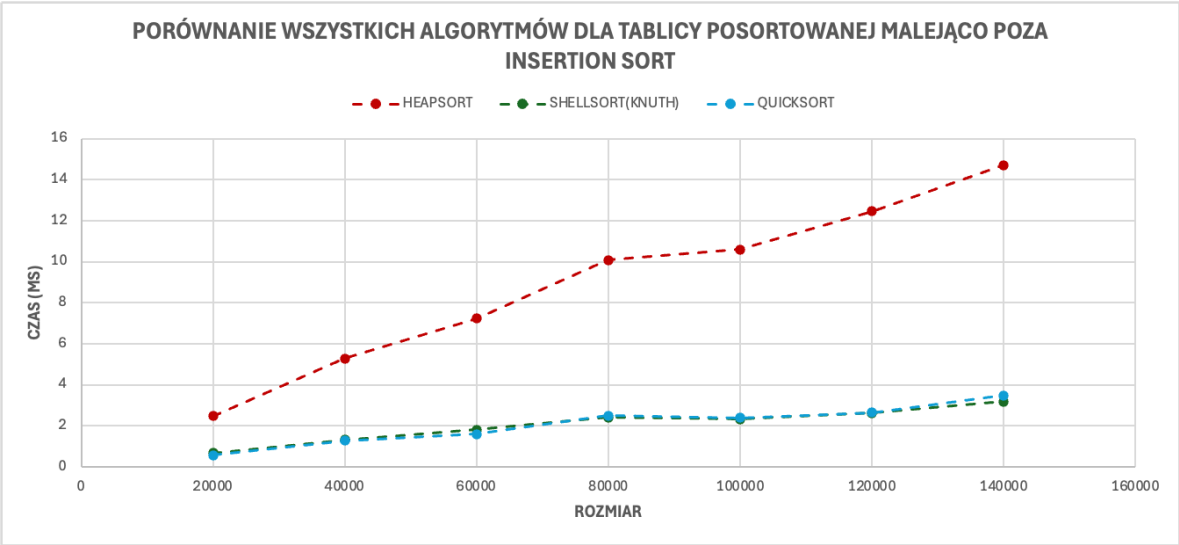
Wykres 44: Wykres czasu sortowania wszystkich algorytmów poza insertion sort w funkcji rozmiaru tablicy - tablica losowa

7.2.3 Dla tablicy uporządkowanej rosnąco



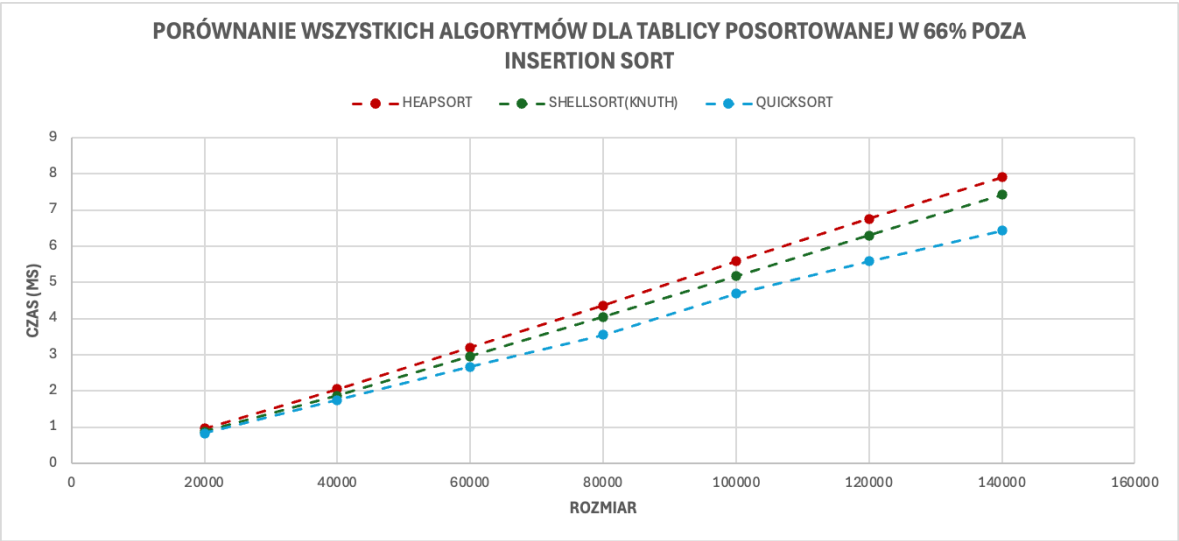
Wykres 45: Wykres czasu sortowania wszystkich algorytmów poza insertion sort w funkcji rozmiaru tablicy - tablica posortowana rosnąco

7.2.4 Dla tablicy uporządkowanej malejąco



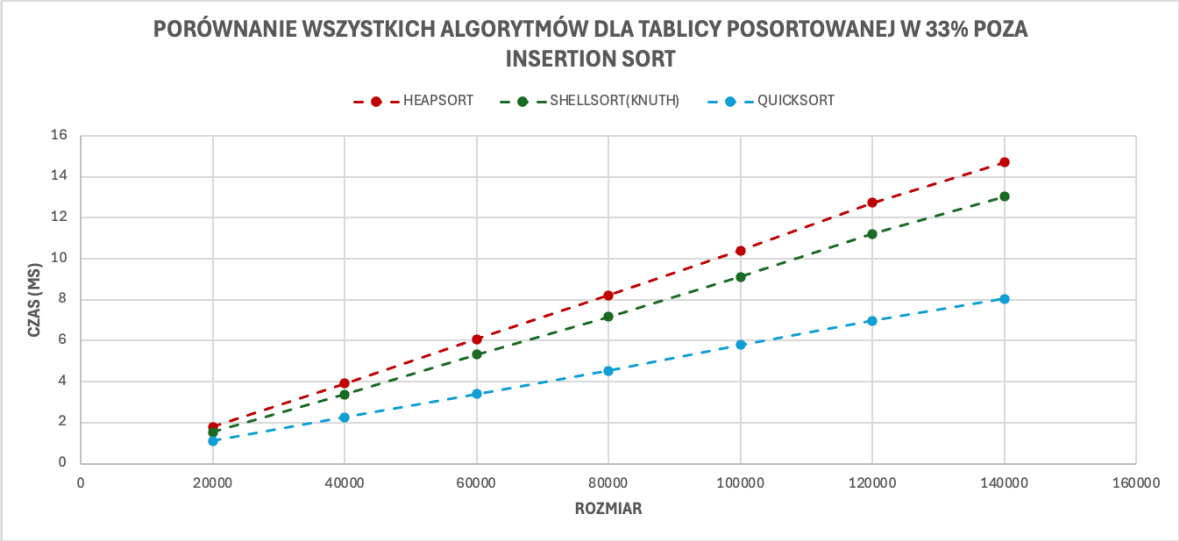
Wykres 46: Wykres czasu sortowania wszystkich algorytmów poza insertion sort w funkcji rozmiaru tablicy - tablica posortowana malejąco

7.2.5 Dla tablicy uporządkowanej w 66%



Wykres 47: Wykres czasu sortowania wszystkich algorytmów poza insertion sort w funkcji rozmiaru tablicy - tablica posortowana w 66%

7.2.6 Dla tablicy uporządkowanej w 33%



Wykres 48: Wykres czasu sortowania wszystkich algorytmów poza insertion sort w funkcji rozmiaru tablicy - tablica posortowana w 33%

7.3 Wnioski

Na podstawie przeprowadzonych testów wydajnościowych można zauważyć różnice pomiędzy poszczególnymi algorytmami w kwestii czasu wykonywania się sortowania jak i również w kwestii odporności na zróżnicowanie danych wejściowych.

- Insertion Sort - najwolniejszy algorytm ze wszystkich przetestowanych w badaniu. Słaba złożoność czasowa $O(n^2)$, algorytm nie radzi sobie z dużymi rozmiarami tablic. Uzyskanymi osiągami czasowymi znacznie odstaje od pozostałej trójki algorytmów. Algorytm radzi sobie dobrze jedynie w momencie gdy wejściowa tablica jest już posortowana lub gdy operuje na małych tablicach. Do zalet algorytmu można zaliczyć stabilność - nie zmienia elementów o tych samych wartościach miejscami.
- Shell Sort - w obu wariantach oferował przyzwoitą wydajność. Odstawał względem quicksorta pod względem czasu sortowania, jednak dobrze sobie radził z utrzymaniem dobrych czasów dla różnych wariantów tablic. Często lepsze czasy od Heap Sorta, dla wariantu tablicy wstępnie posortowanej osiągnął drugi najlepszy czas po Insertion Sorcie
- Quick Sort - zdecydowanie najlepsze osiągi pod względem czasu sortowania na tablicach losowych i częściowo uporządkowanych. Wybór pivota jest kluczowy w przypadku tego algorytmu dla uzyskania wysokiej wydajności. Dla bardzo dużych rozmiarów tablic występowało ryzyko przepełnienia stosu.
- Heap Sort - niezależnie od danych wejściowych zawsze bardzo zbliżony czas sortowania. Pomimo bycia nieco wolniejszym w działaniu algorytm ten oferuje wysoką odporność na niestandardowe dane wejściowe czyniąc go bardzo uniwersalnym

8 Literatura

Źródła

[1] Thomas H. Cormen, Charles E. Leiserson, and Ronald L. Rivest. *Wprowadzenie do algorytmów*. s. 187. MIT Press / WNT, 1997.

[2] Wikipedia - article authors. *Insertion sort — Wikipedia, The Free Encyclopedia*. 2025. URL: `https://en.wikipedia.org/wiki/Insertion_sort`.

[3] Wikipedia - autorzy artykułu. *Sortowanie Shella — Wikipedia, wolna encyklopedia*. 2025. URL: `https://pl.wikipedia.org/wiki/Sortowanie_Shella`.

[4] Wikipedia - autorzy artykułu. *Sortowanie szybkie — Wikipedia, wolna encyklopedia*. 2025. URL: `https://pl.wikipedia.org/wiki/Sortowanie_szybkie`.

[5] WsCube Tech. *Heap Sort - Data Structures and Algorithms (DSA)*. 2025. URL: `https://www.wscubetech.com/resources/dsa/heap-sort`.

[6] WsCube Tech. *Shell Sort - Data Structures and Algorithms (DSA)*. 2025. URL: `https://www.wscubetech.com/resources/dsa/shell-sort`.